Nickel and Diming Your GAN: A Dual-Method Approach to Enhancing GAN Efficiency via Knowledge Distillation

엣지컴퓨팅 논문 발표 B조

자동차it융합학부 장예지

소프트웨어학부 이가현

소프트웨어학부 강하운

Introduction

GAN + Compression

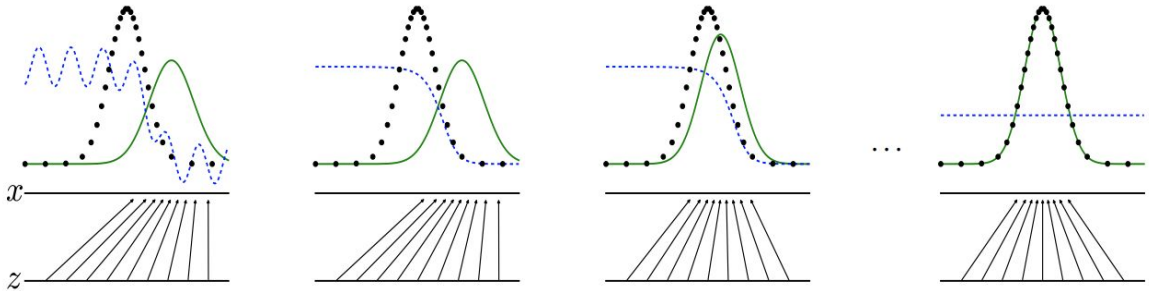
GAN

What is GAN?

1. **‘Adversarial’**과정을 통해 생성 모델을 구축하는 새로운 프레임 워크

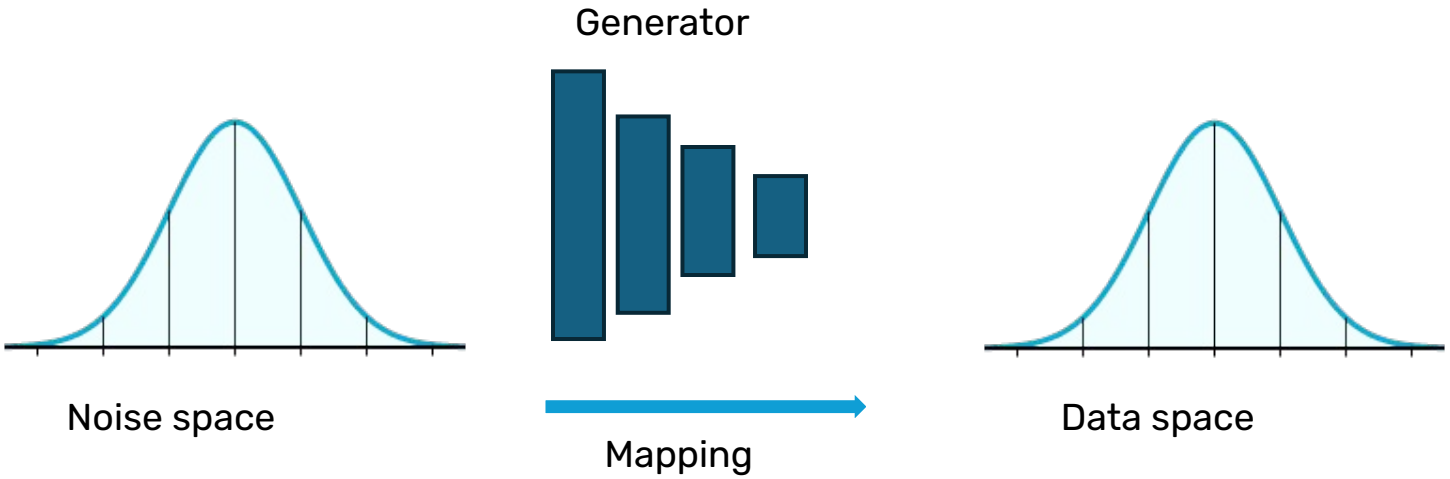
2. **Two** network

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]$$



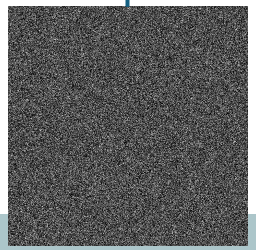
GAN

Generator 관점에서

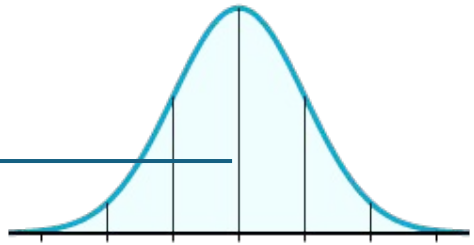


GAN

Generator 관점에서

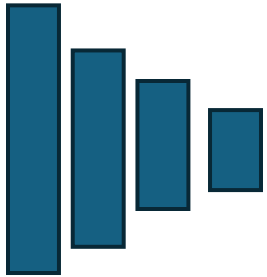


noise

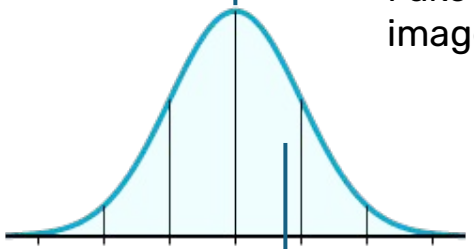


Noise space

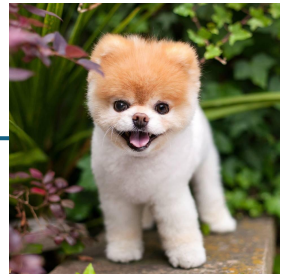
Generator



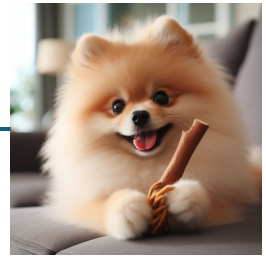
Mapping



Data space



Fake image

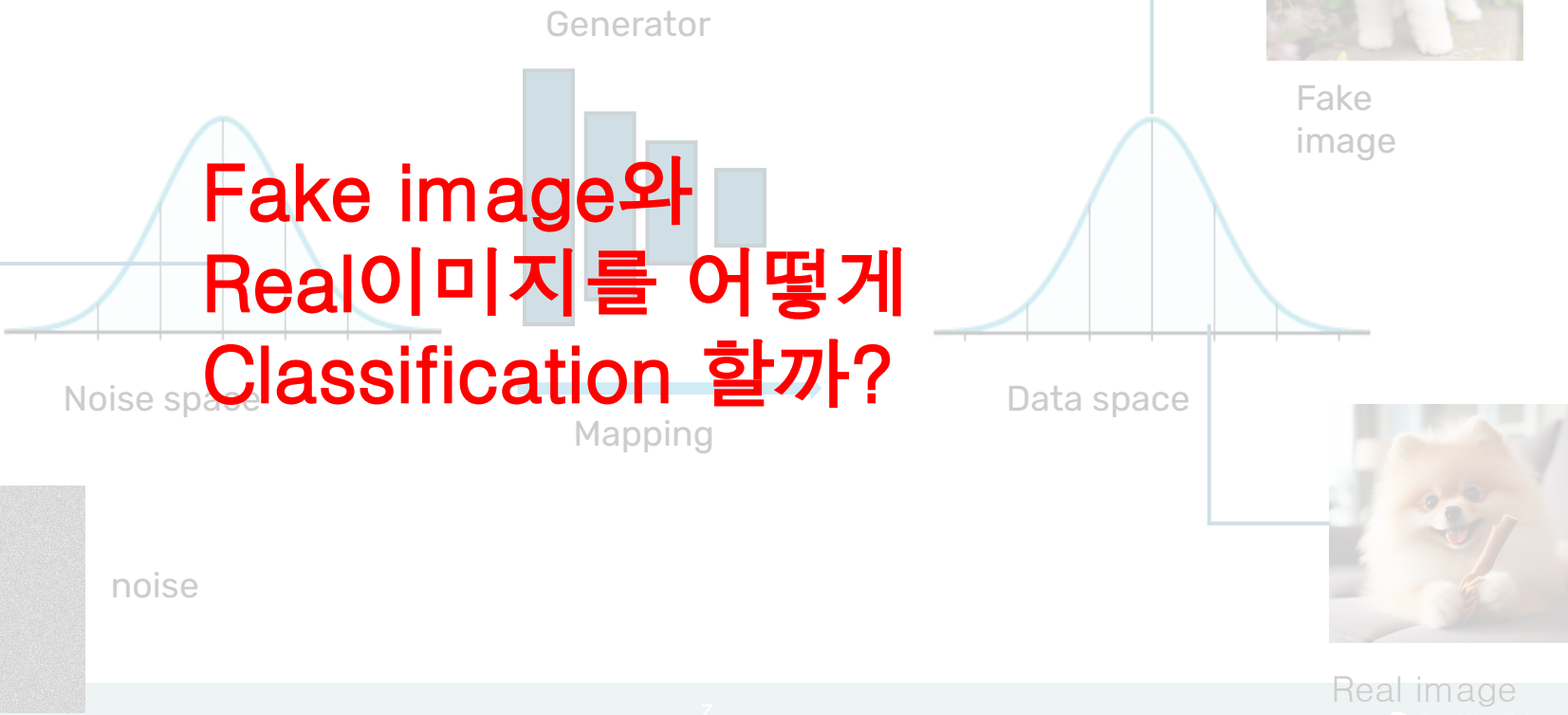


Real image

국민대학교 엣지컴퓨팅 B조

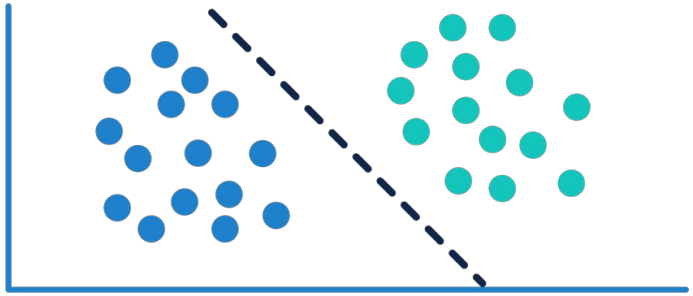
GAN

Generator 관점에서

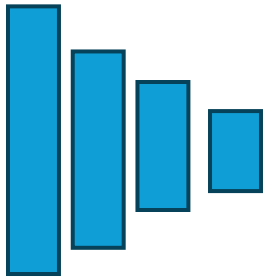


Fake image와
Real이미지를 어떻게
Classification 할까?

GAN



전통적인 방법은 매우 **heuristic**함
생성모델의 **성능이 저하됨**



Discriminator

만약 classification을 위해 **하나의 네트워크** 를 더 추가한다면
image classification의 **유연성** 을 얻을 수 있음

GAN

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]$$

Discriminator: fake 이미지와 real 이미지 Loss차이 극대화

Generator: fake 이미지와 real 이미지 Loss차이 극소화

GAN

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]$$

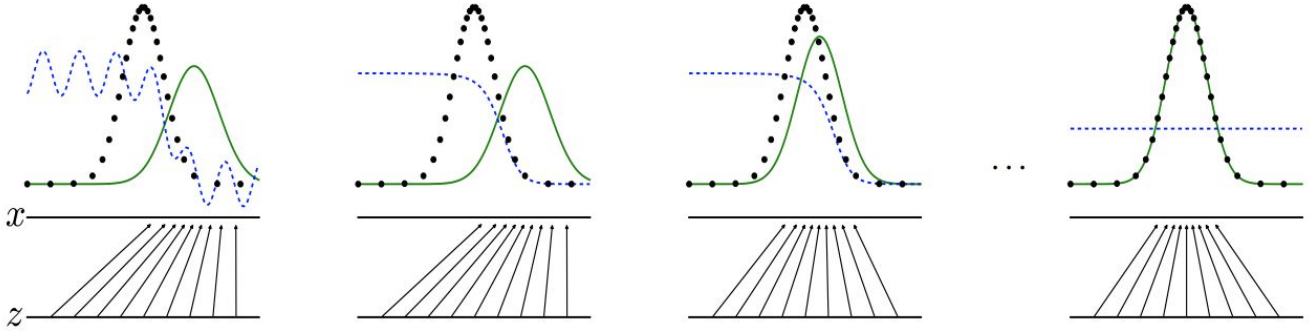
Discriminator: fake 이미지와 real 이미지 Loss차이 극대화

Generator: fake 이미지와 real 이미지 Loss차이 극소화



Generate **Adversarial** Network

GAN



점선 분포: Data영역 분포
초록 분포: Generator 분포
파란 분포: Discriminator 분포

Generator=> Data 분포와 비슷한 분포를 갖는 것이 목표!

GAN

장점

더 유연한 classification
더 좋은 성능

단점

두 network를 한 training안에 사용하여 **unstable**
많은 **연산량**

GAN

장점

더 유연한 classification
더 좋은 성능

단점

두 network를 한 training안에 사용하여 **unstable**  Compression 실험 목표
많은 **연산량**

GAN

실험 목표

더 **안정적**이고 **성능이 좋은** GAN Compression 방법을 찾는것

More detail

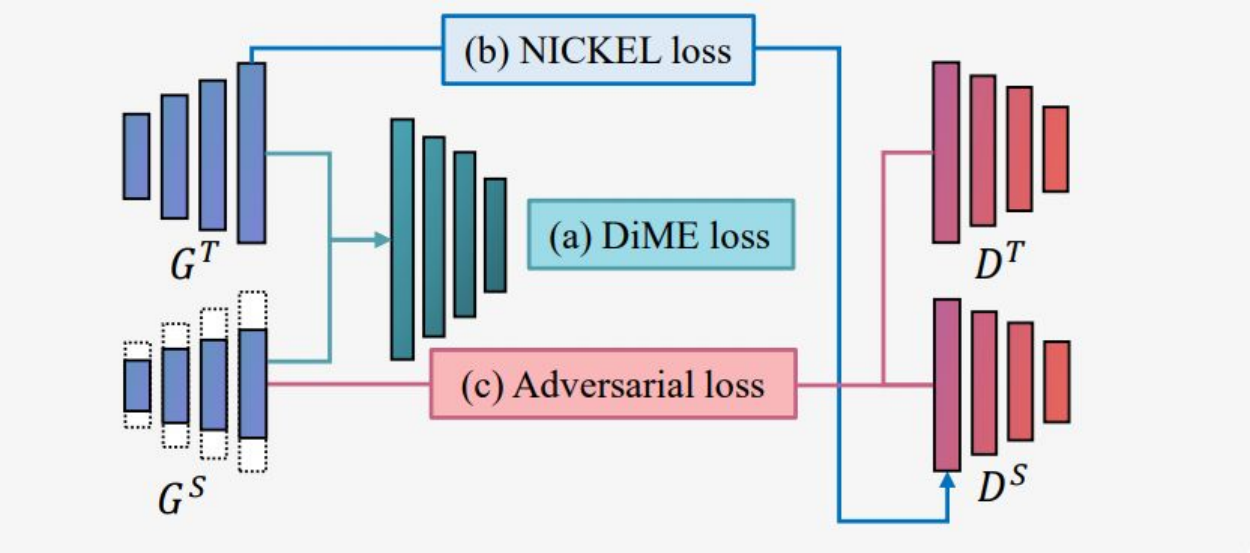
우리가 고려해야할 요소

(1) GAN Compression의 구조

(2) Discriminator & Generator Balance

(3) GGDR

What we consider

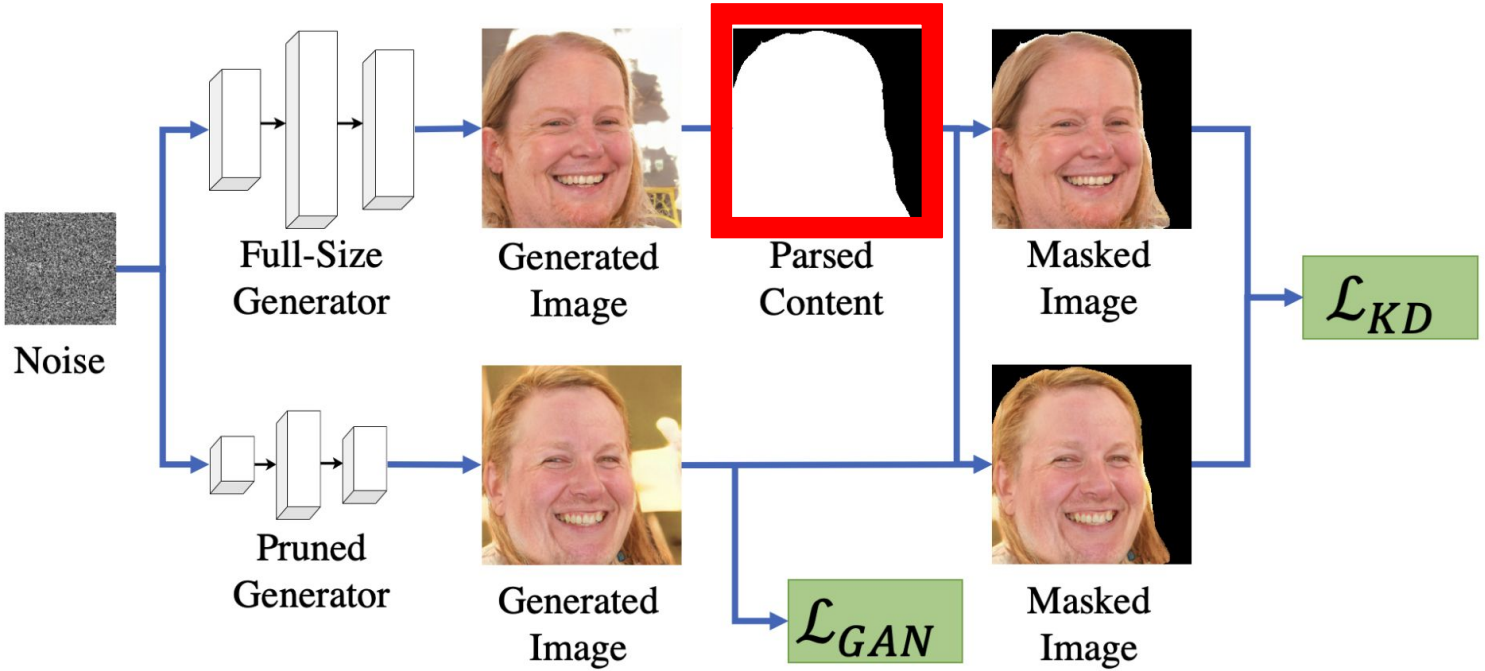


compression GAN
Distillation은 4개의 network가 존재함

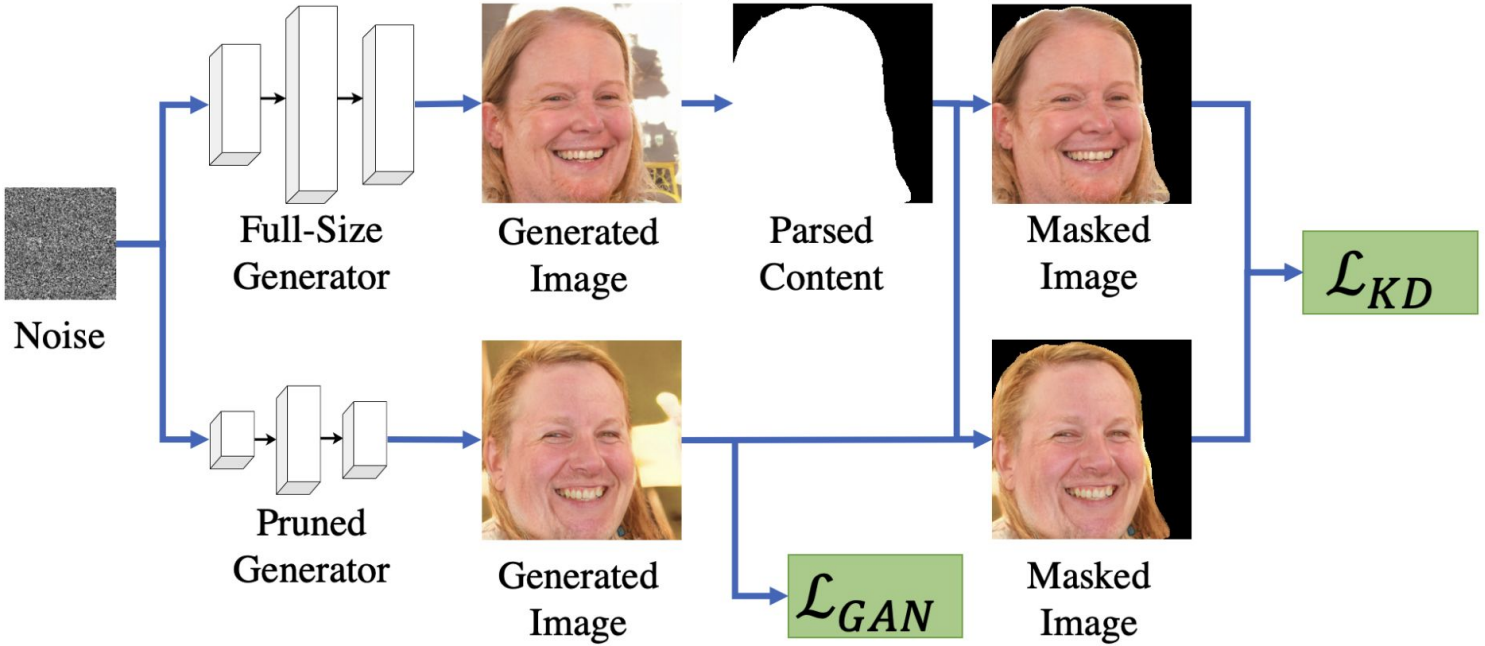
Teacher(full-model):
generator & discriminator

Student(compression):
Generator & discriminator

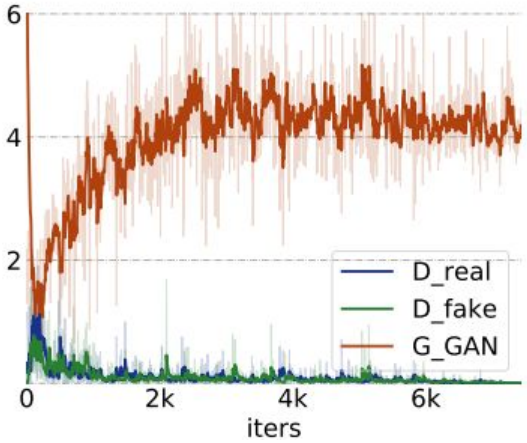
What we consider



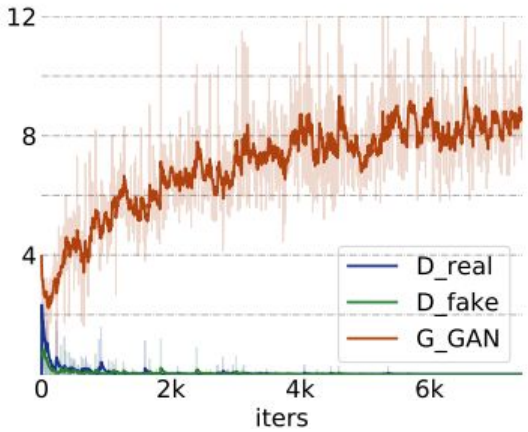
What we consider



What we consider

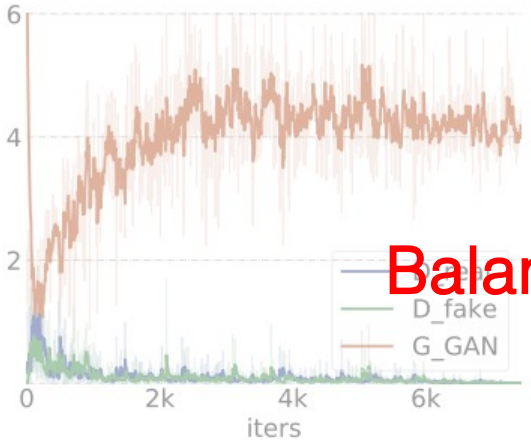


Original G + Original D Loss



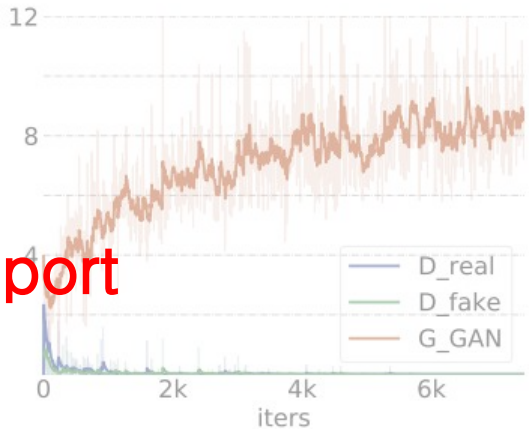
¼ channel width G + Original D Loss

What we consider



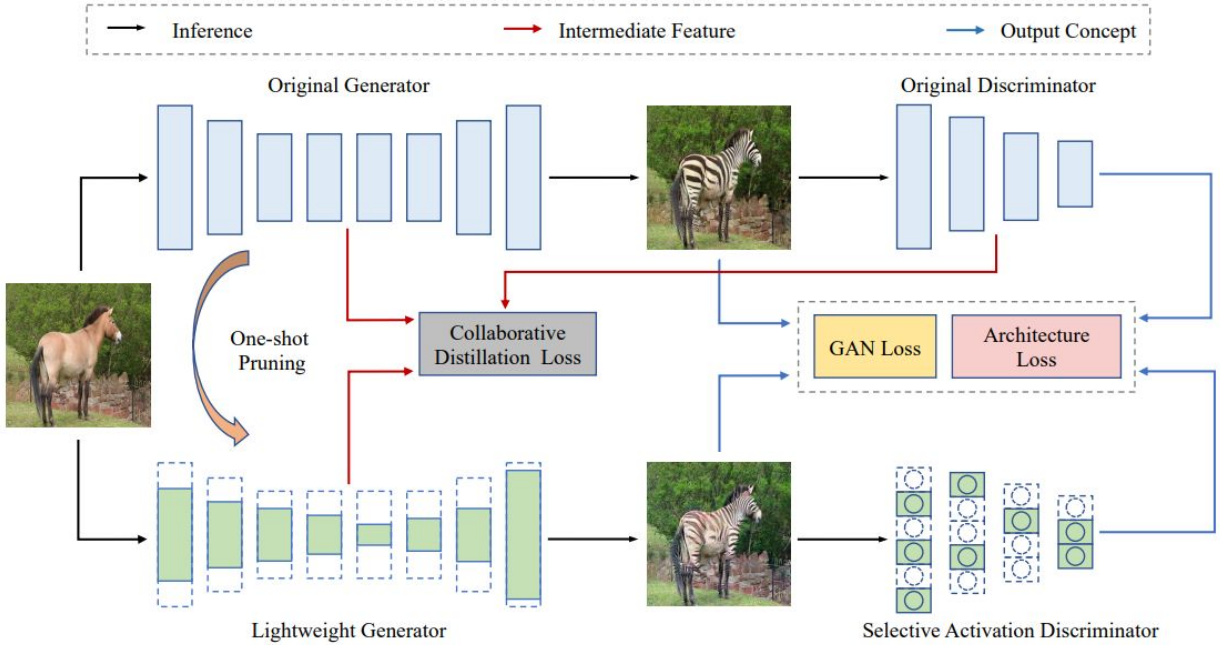
Original G + Original D Loss

Balance import



¼ channel width G + Original D Loss

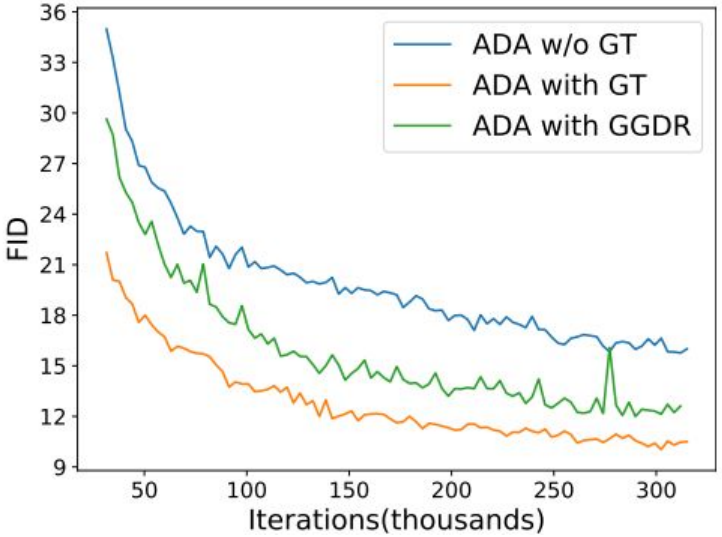
What we consider



Generator가 compression하는 동안, Discriminator 또한 **실험적인 비율**로 compression함

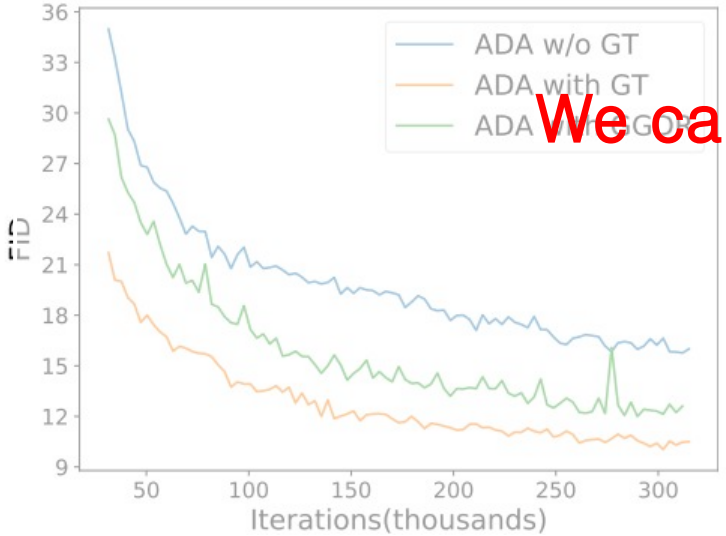
Discriminator가 Adversarial 이외에도
Generator에서 Distillation을 하는
역할로 사용할 수 있을까?

What we consider



이러한 방법이 좋은 성능을
이끈다는 것을 입증함

What we consider



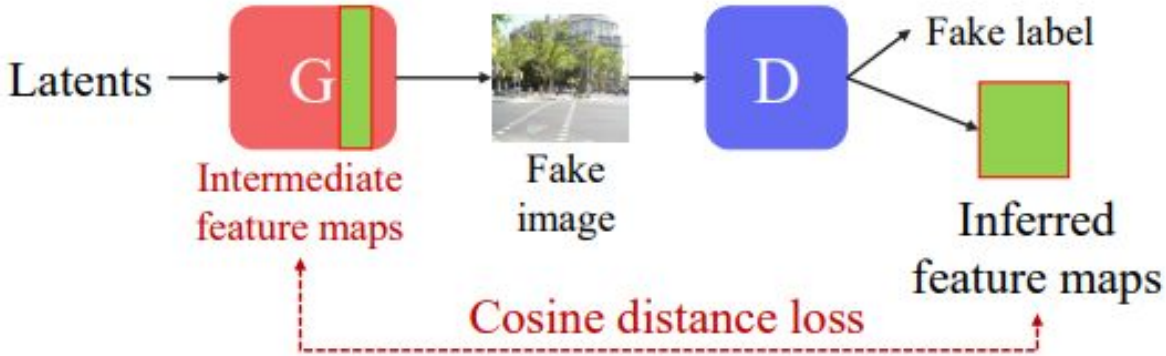
We called GGDR

이러한 방법이 좋은 성능을
이끈다는 것을 입증함

What we consider

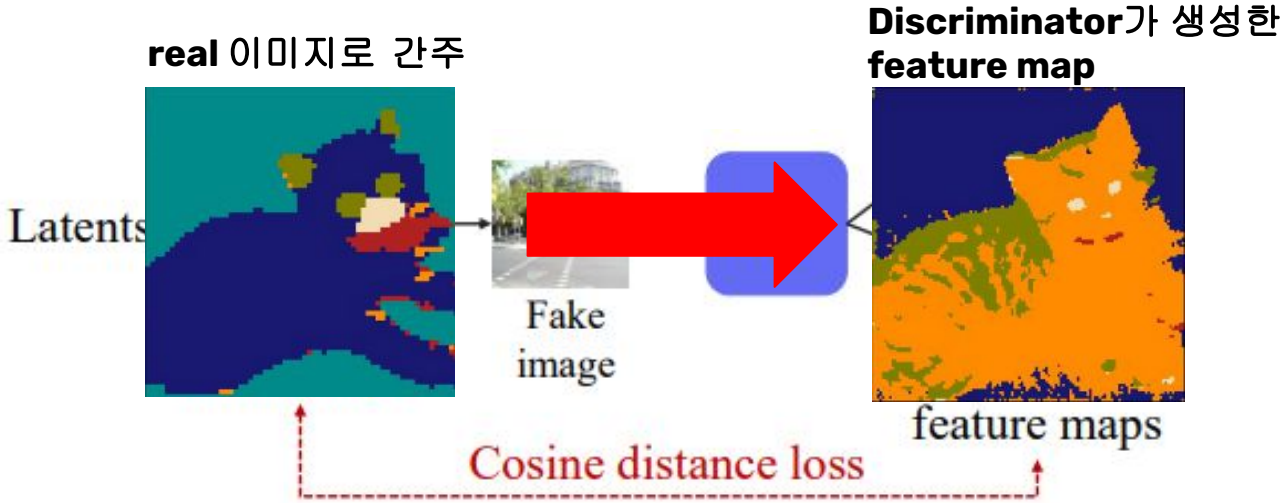
$$\max_D \mathcal{L}_{ggdr}(G, D) =$$

$$- \mathbb{E}_{z \sim p(z)} \left[1 - \frac{F^t(G(z)) \cdot G(z)^t}{\|F^t(G(z))\|_2 \cdot \|G(z)^t\|_2} \right]$$

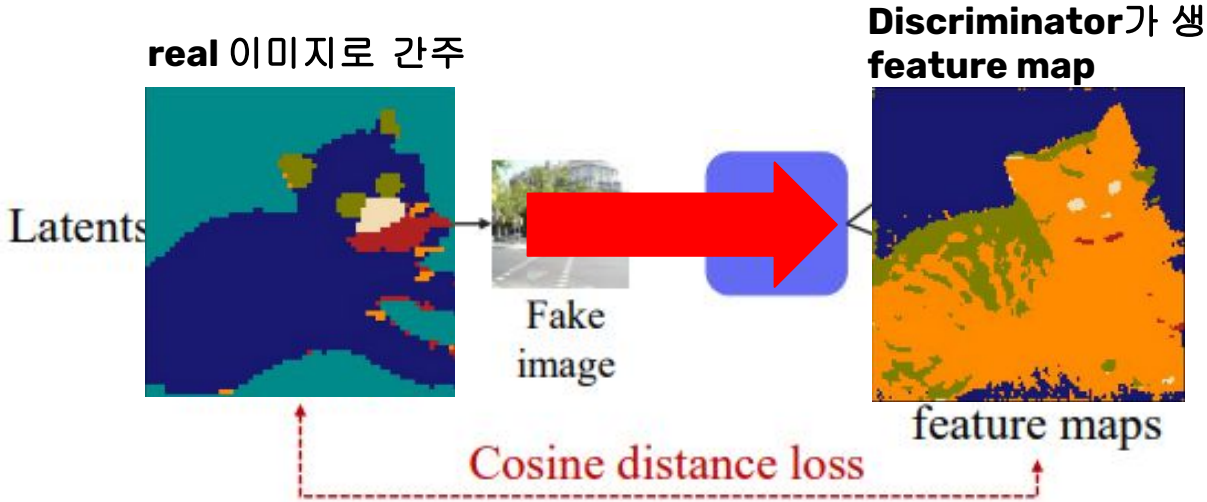


Generator의 **featuremap**을 **real** 이미지 로
간주하여 Discriminator로 전달하여
Discriminator를 Distillation하는 방법

What we consider



What we consider

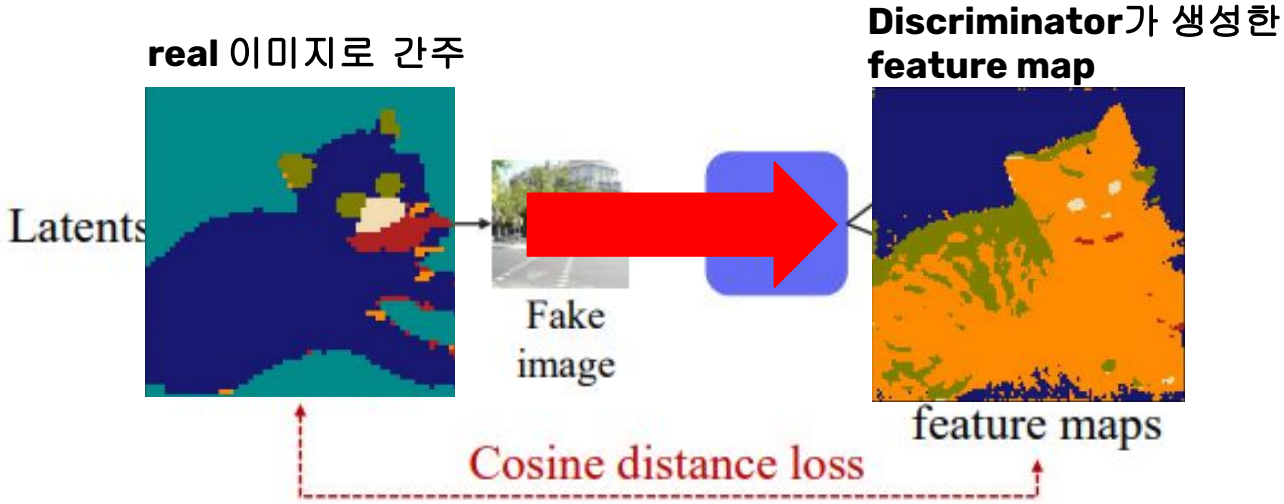


Discriminator=> adversarial loss뿐만 아닌 Distillation 또한 진행됨

$$\max_D \mathcal{L}_{ggdr}(G, D) =$$

$$- \mathbb{E}_{z \sim p(z)} \left[1 - \frac{F^t(G(z)) \cdot G(z)^t}{\|F^t(G(z))\|_2 \cdot \|G(z)^t\|_2} \right]$$

What we consider



Discriminator=> adversarial loss뿐만 아닌 Distillation 또한 진행됨

$$\max_D \mathcal{L}_{ggdr}(G, D) = - \mathbb{E}_{z \sim p(z)} \left[1 - \frac{F^t(G(z)) \cdot G(z)^t}{\|F^t(G(z))\|_2 \cdot \|G(z)^t\|_2} \right]$$

How to

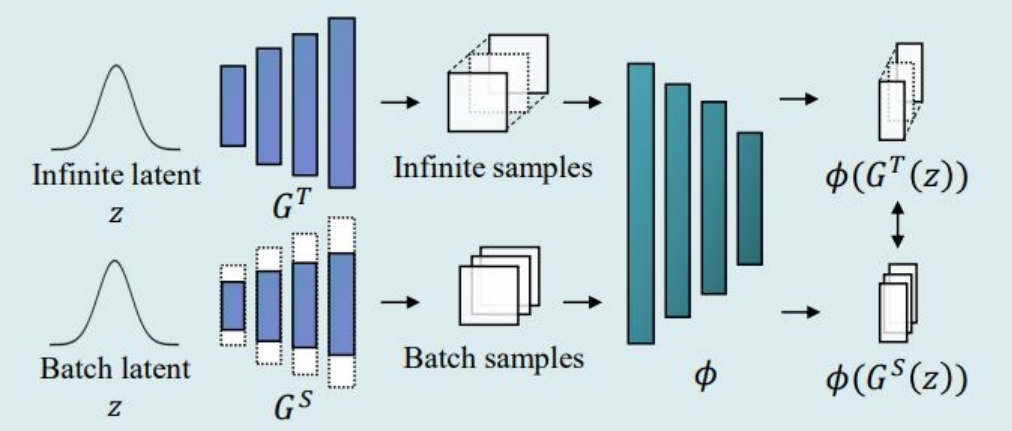
어떻게 우리 모델 Compression을 진행 하는지

(1) DiME

(2) NICKEL

How to

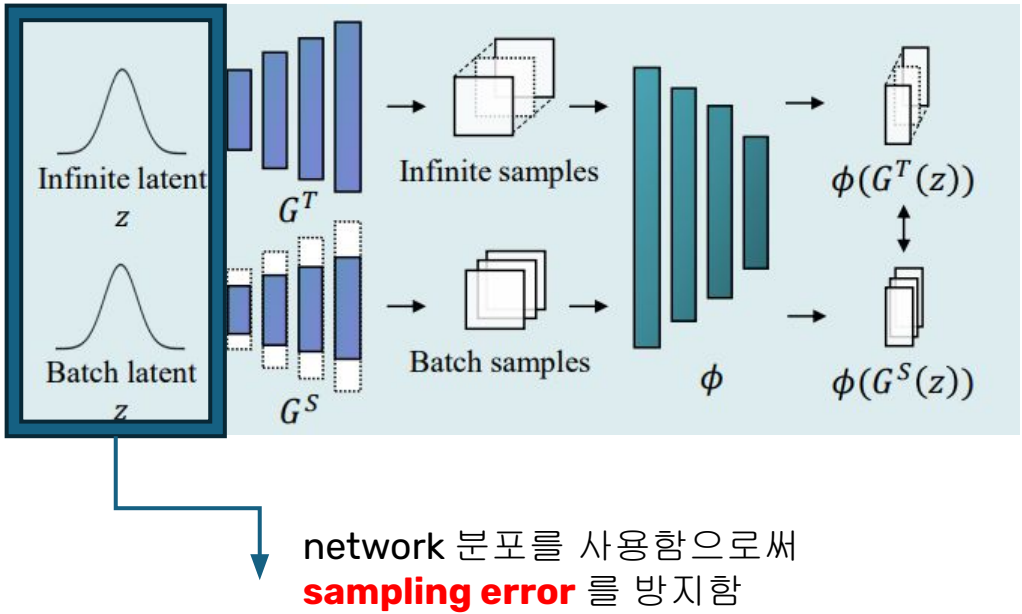
Distribution Matching



파운데이션 모델을 **kernel** 로 사용함으로써
이러한 두 **독립적인** 분포 간 거리를 구할 수 있다.

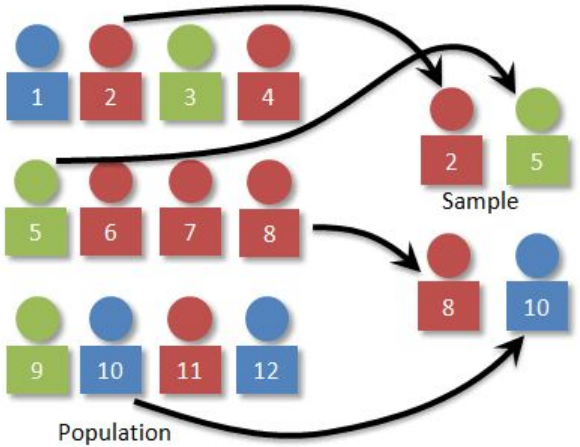
How to

Distribution Matching



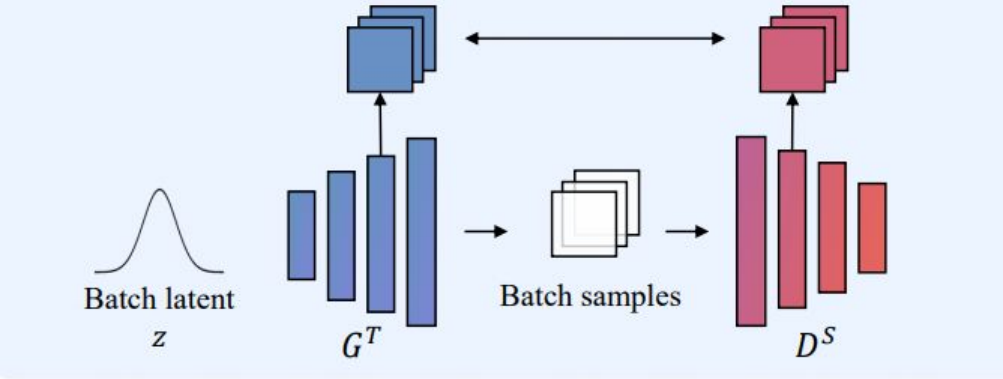
network 분포를 사용함으로써
sampling error 를 방지함

파운데이션 모델을 **kernel** 로 사용함으로써
이러한 두 **독립적인** 분포 간 거리를 구할 수 있다.



How to

Distillation Exchange

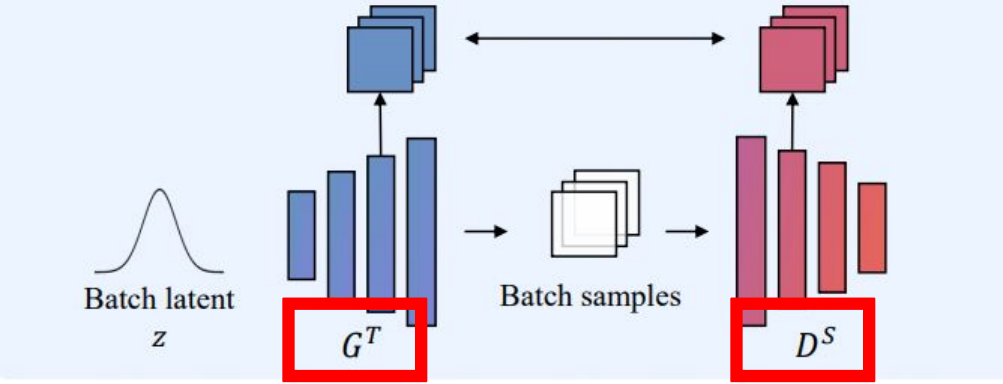


GGDR에서 영감을 받아 Gt에서 Ds로 Distillation을 함

GGDR과 달리 초기 학습단계에서 **stable**함

How to

Distillation Exchange

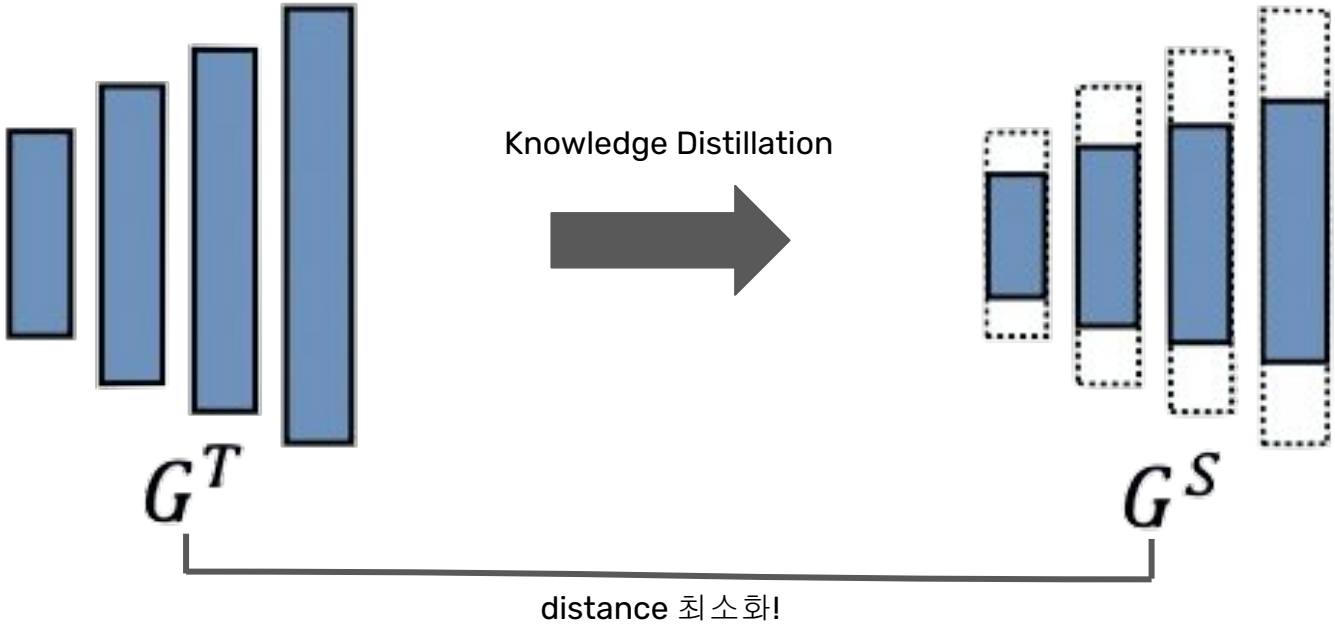


GGDR에서 영감을 받아 Gt에서 Ds로 Distillation을 함

GGDR과 달리 초기 학습단계에서 **stable**함

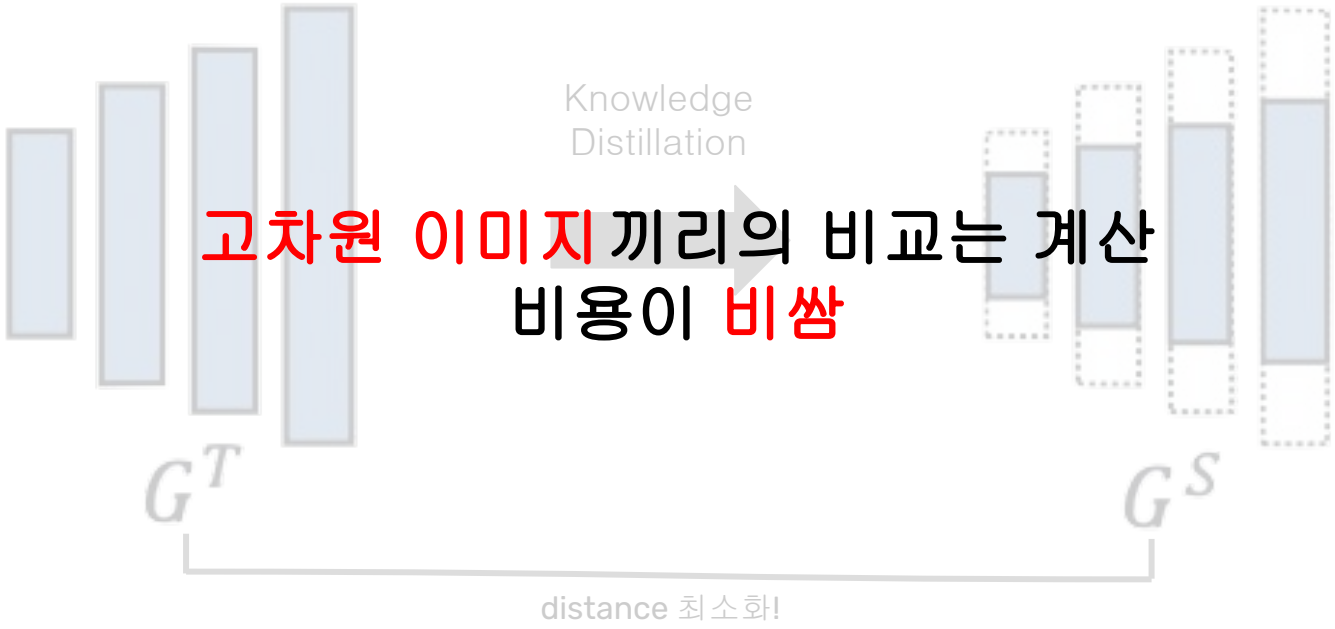

DiME

DiME



Gs가 Gt를 충분히 모방하기 위해서는 Gt와 Gs의 distance를 최소화해야 함

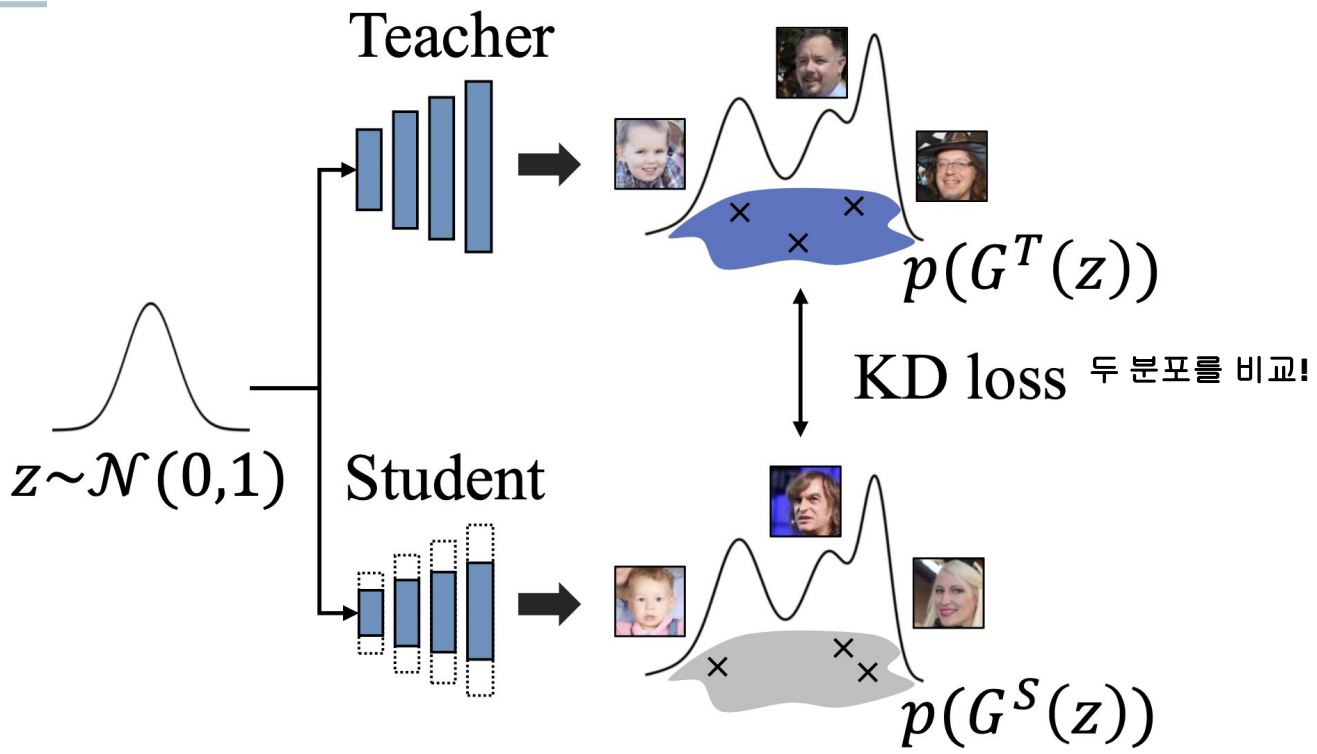
DiME



고차원 이미지끼리의 비교는 계산 비용이 비쌈

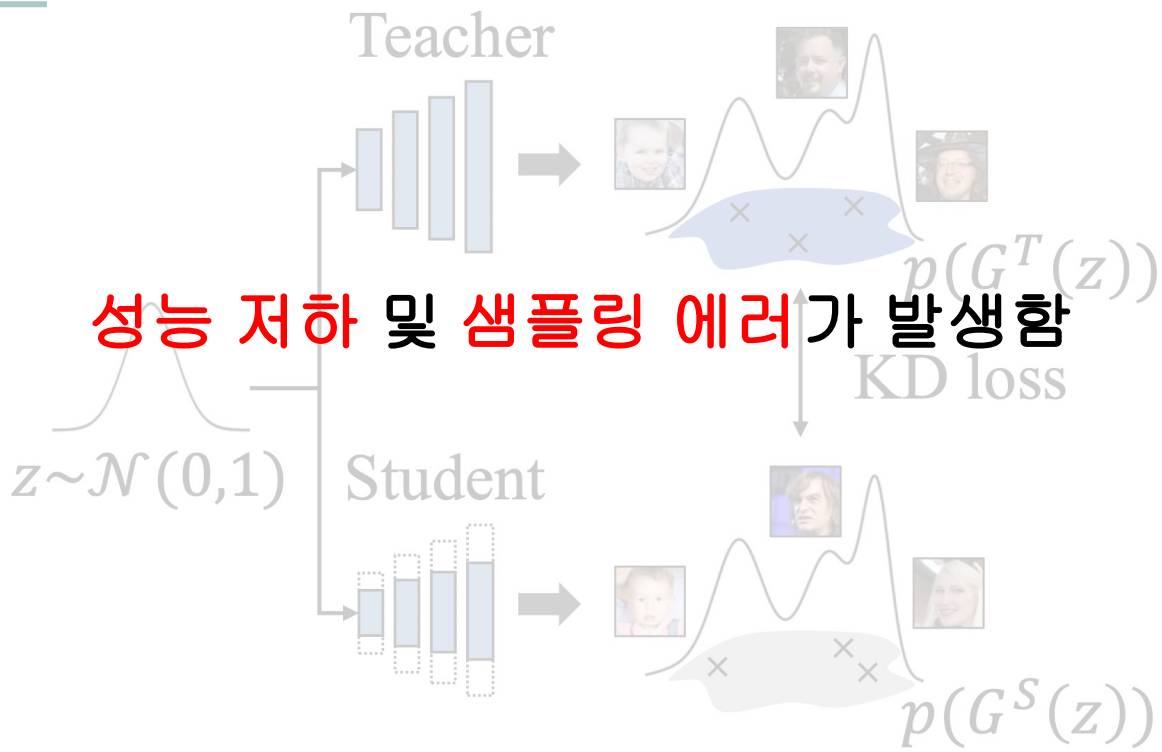
Gs가 Gt를 충분히 모방하기 위해서는 Gt와 Gs의 distance를 최소화해야 함

DiME



따라서 대부분의 GAN Compression 방법들을 분포를 비교하는 방식을 선택함

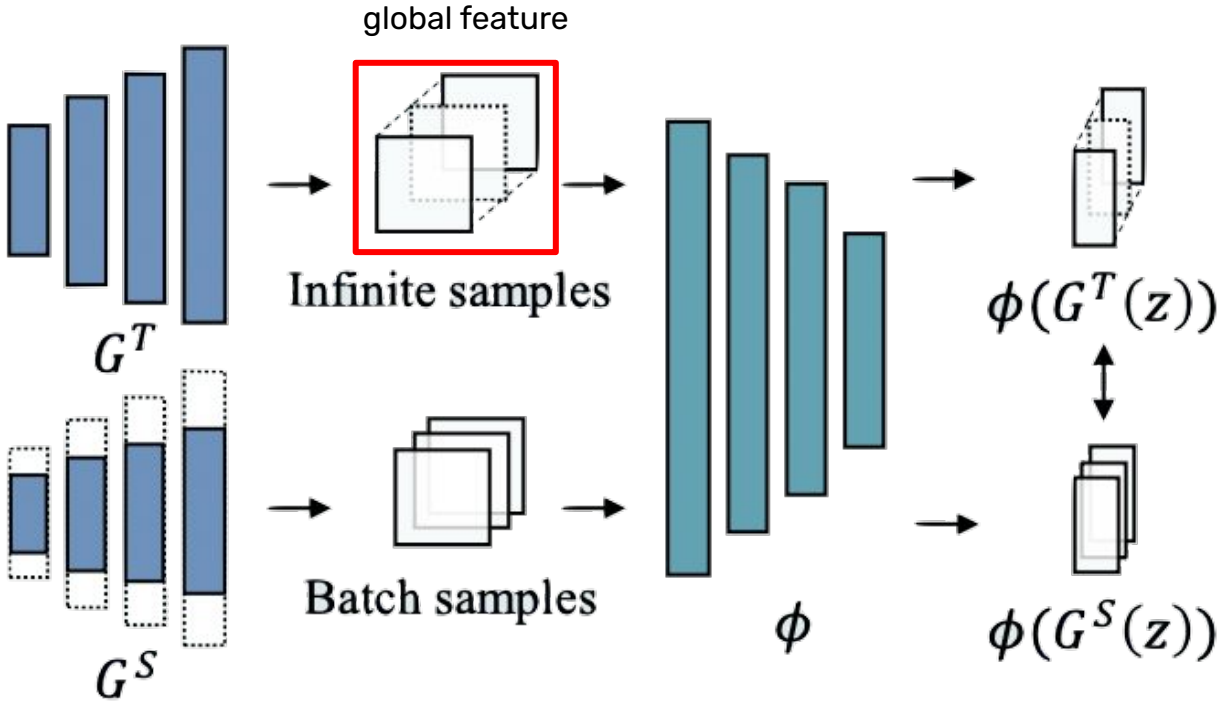
DiME



성능 저하 및 샘플링 에러가 발생함

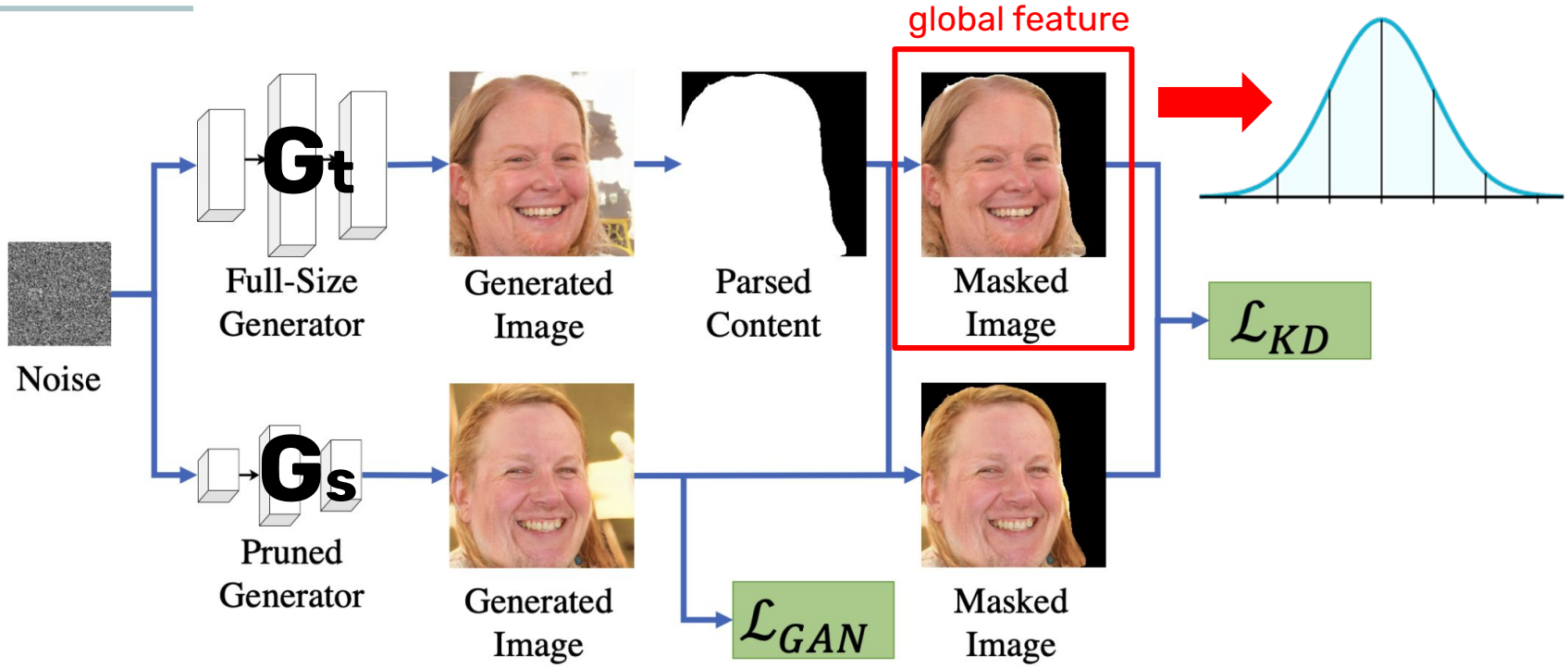
따라서 대부분의 GAN Compression 방법들을 분포를 비교하는 방식을 선택함

DiME



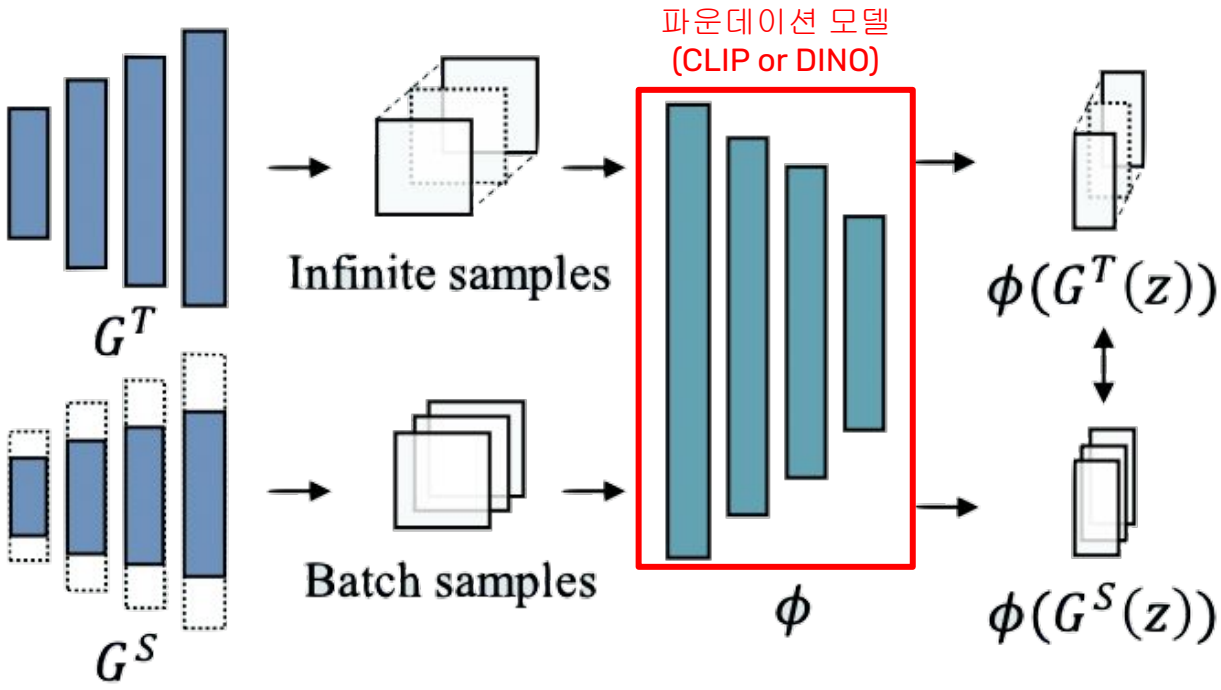
pre-trained 모델인 Gt의 global feature를 사용하면 샘플링 에러를 줄일 수 있음

DiME



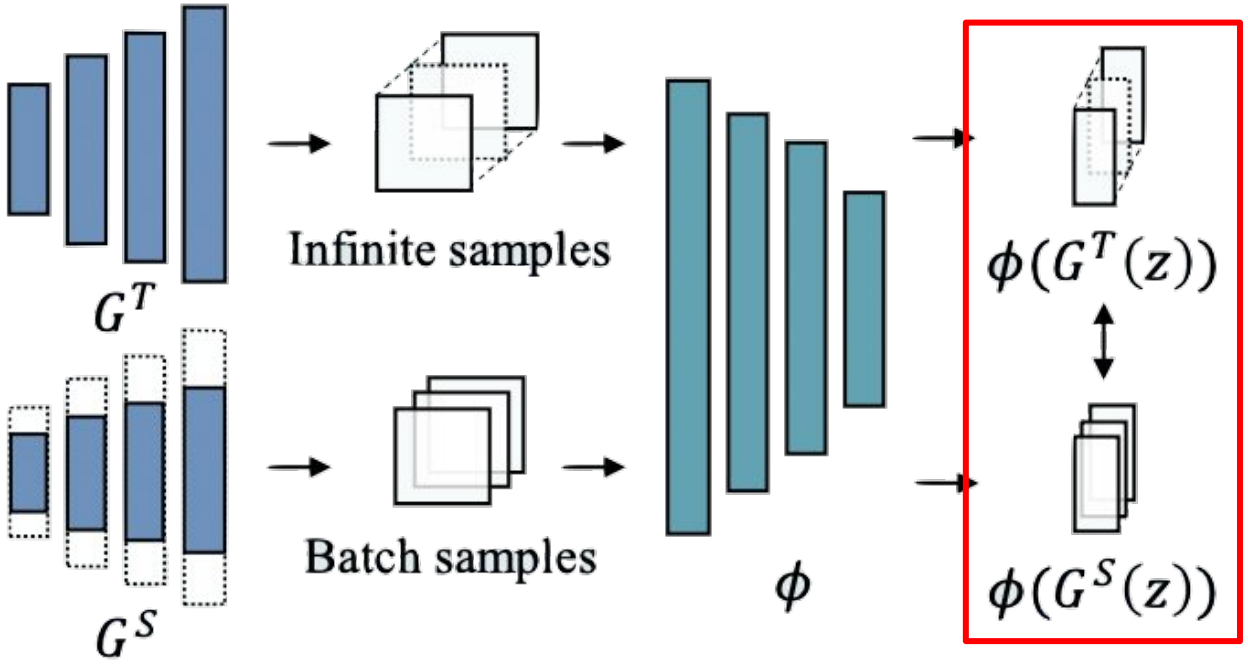
Gt의 global feature를 분포 형태로 사용하면, Gs가 중요한 feature에 대해서 잘 학습하게 됨

DiME



파운데이션 모델(CLIP or DINO)을 임베딩 커널 ϕ로 사용함

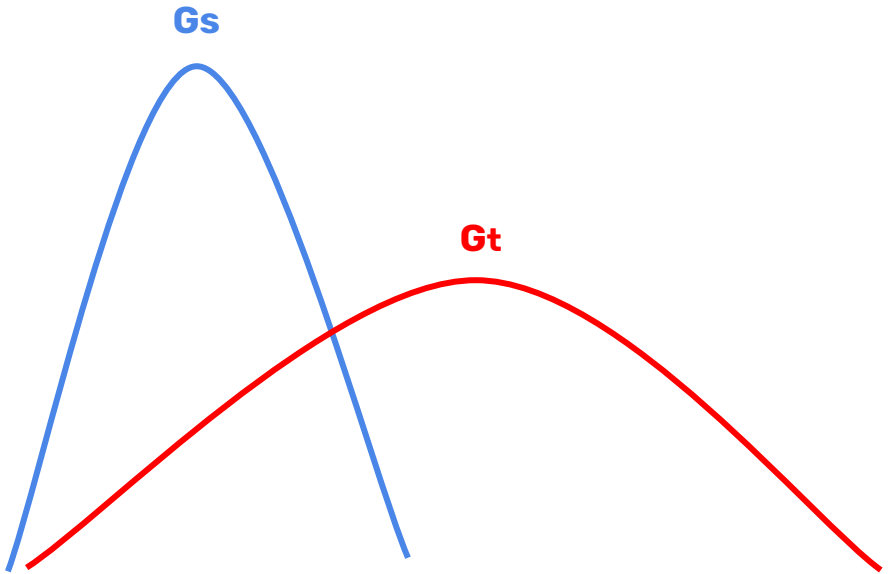
DiME

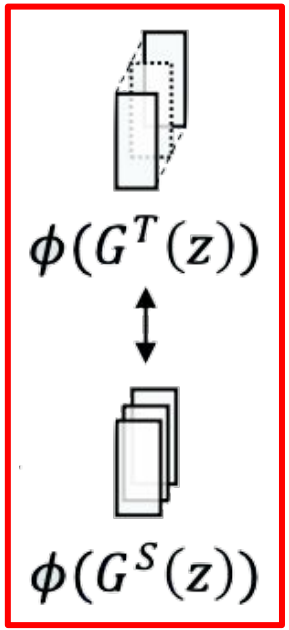


MMD를 이용해
두 분포를 비교!

Gt와 Gs의 분포를 비교하여, 두 분포가 일치되도록 학습함

DiME





MMD를 이용해
두 분포를 비교!

이때 두 분포를 일치시키는 방법으로 MMD loss를 사용함

DiME

$$G^T(z)$$

DiME

$$\phi(G^T(z))$$

DiME

$$\phi(G^T(z)), \phi(G^S(z))$$

DiME

$$\mathbb{E}[||\phi(G^T(z)), \phi(G^S(z))||_1]$$

Gt와 Gs 분포의 평균 임베딩 차이 (MMD loss)

DiME

$$\hat{\mathcal{D}}_l^{mmd} = \frac{1}{n_s^2}\sum_{i=1}^{n_s}\sum_{j=1}^{n_s} k_l(\phi_l(\boldsymbol{x}_i^s), \phi_l(\boldsymbol{x}_j^s)) + \frac{1}{n_t^2}\sum_{i=1}^{n_t}\sum_{j=1}^{n_t} k_l(\phi_l(\boldsymbol{x}_i^t), \phi_l(\boldsymbol{x}_j^t)) - \frac{2}{n_s n_t}\sum_{i=1}^{n_s}\sum_{j=1}^{n_t} k_l(\phi_l(\boldsymbol{x}_i^s), \phi_l(\boldsymbol{x}_j^t))$$

MMD

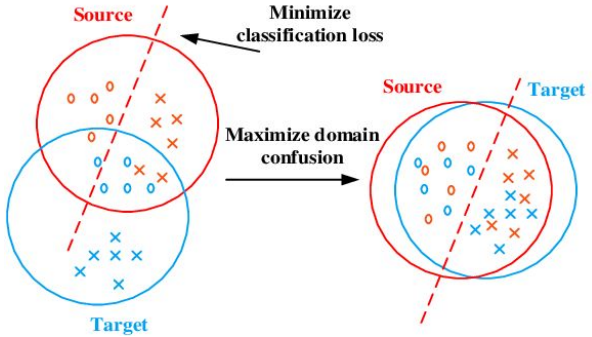
-> Gt 분포의 분산

-> Gs 분포의 분산

-> Gt와 Gs의 유사도

$$\mathbb{E}[||\phi(G^T(z)), \phi(G^S(z))||_1]$$

Gt와 Gs 분포의 평균 임베딩 차이 (MMD loss)



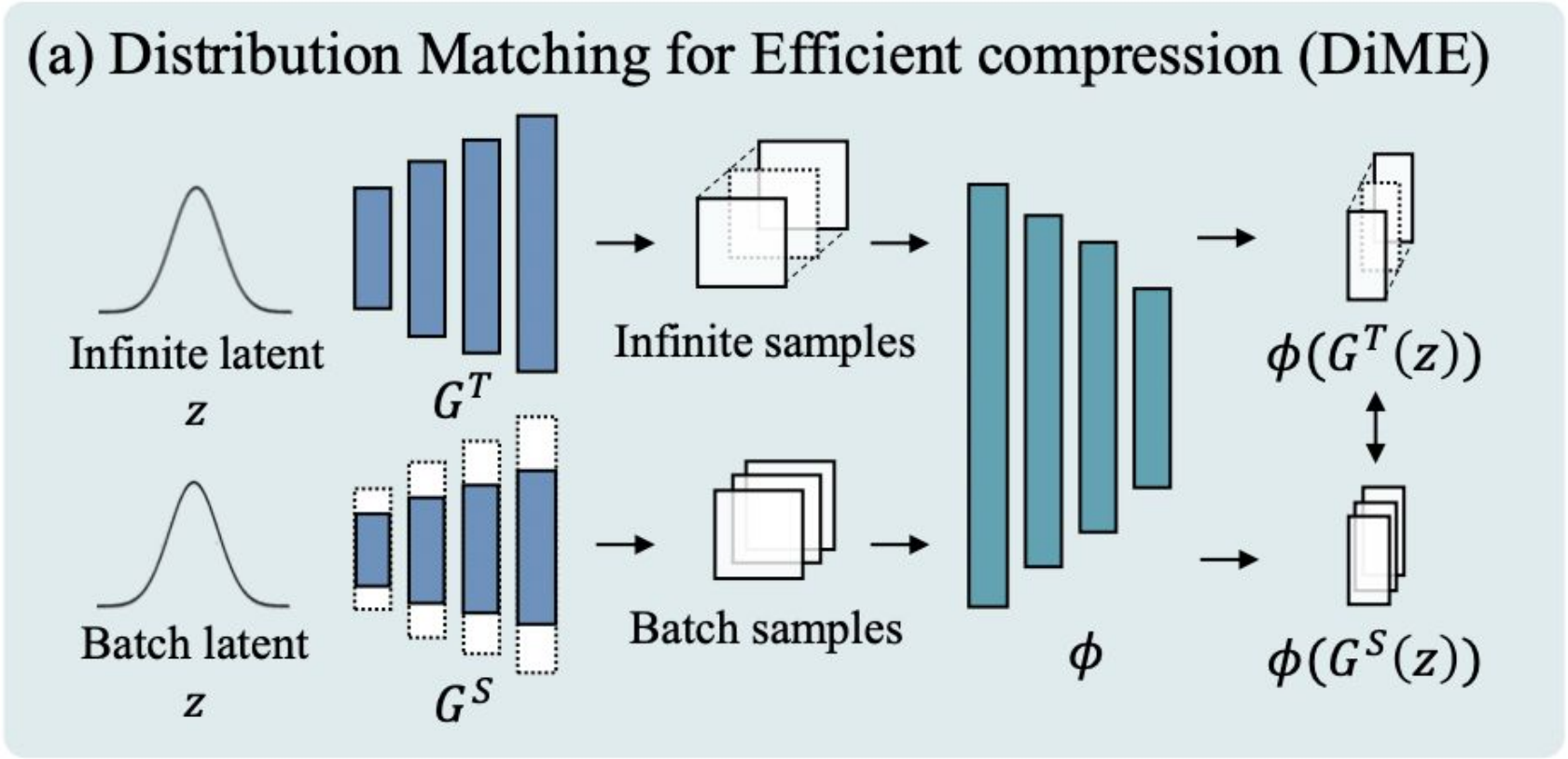
DiME

$$\mathcal{L}_{KD} = d_{kd}(G^T(z), G^S(z)) = \mathbb{E}[||\phi(G^T(z)), \phi(G^S(z))||_1]$$

두 Generator 간의 distance

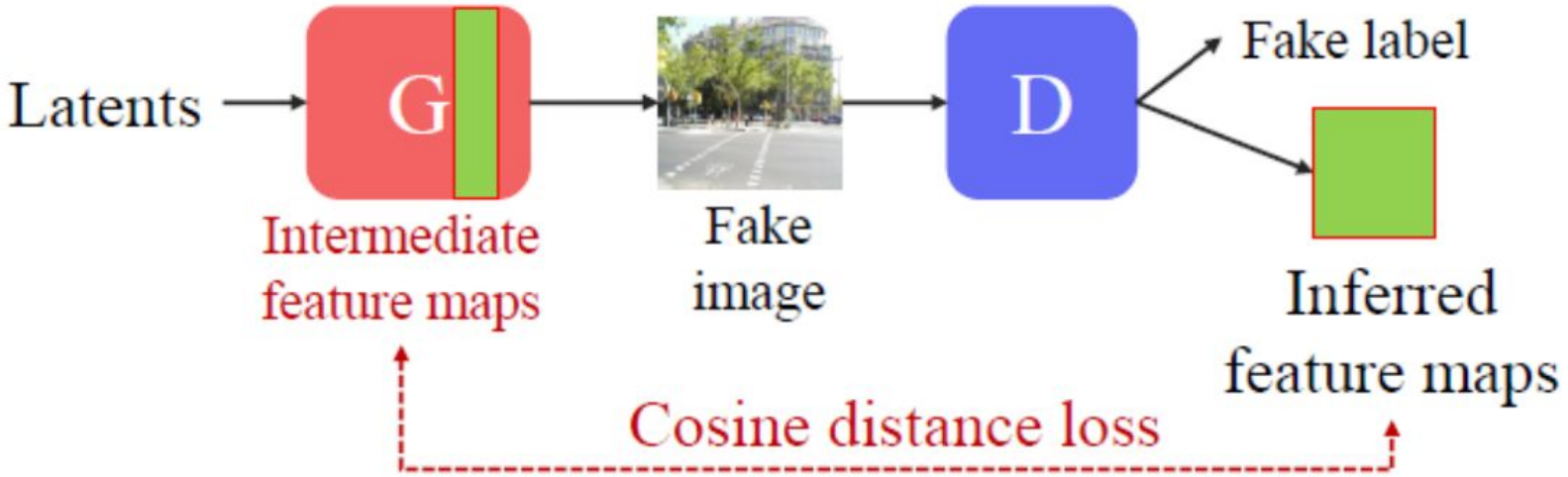
Gt와 Gs 분포의 평균 임베딩 차이 (MMD loss)

DiME


(a) Distribution Matching for Efficient compression (DiME)
Infinite latent
z
G^T
Infinite samples
$\phi(G^T(z))$
Batch latent
z
G^S
Batch samples
ϕ
$\phi(G^S(z))$

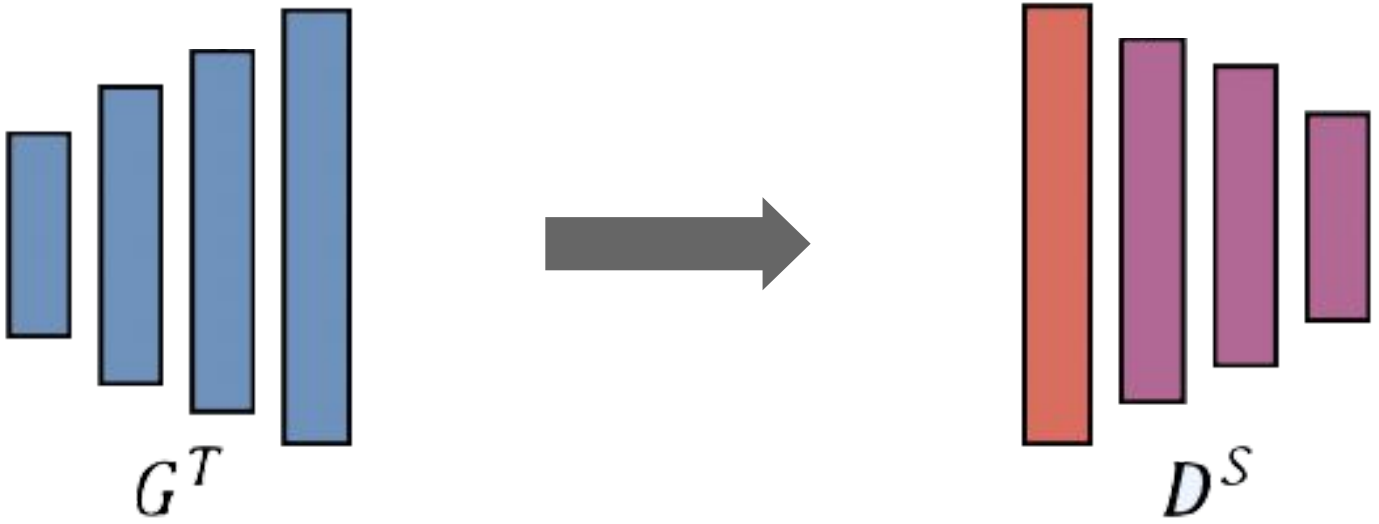
NICKEL.

GGDR



G -> D로 지식을 증류하여 학습하면 D의 성능을 끌어올릴 수 있음

NICKEL



NICKEL은 Gt -> Ds로 지식을 증류하여 학습하면 Ds의 성능을 끌어올리고자 함

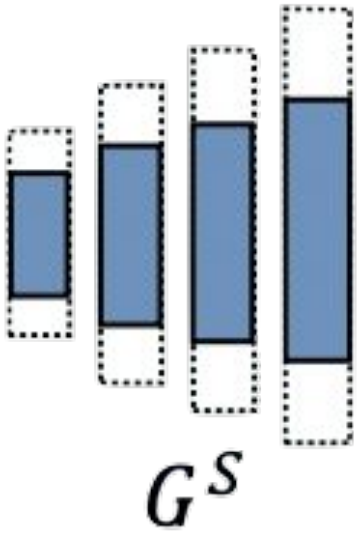
NICKEL

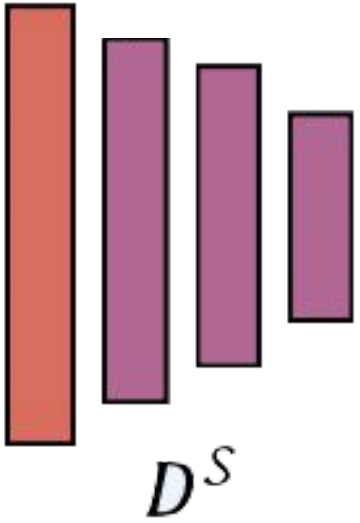


왜 Gs가 아니라 Gt를 사용하였을까?

NICKEL은 Gt -> Ds로 지식을 증류하여 학습하면 Ds의 성능을 끌어올리고자 함

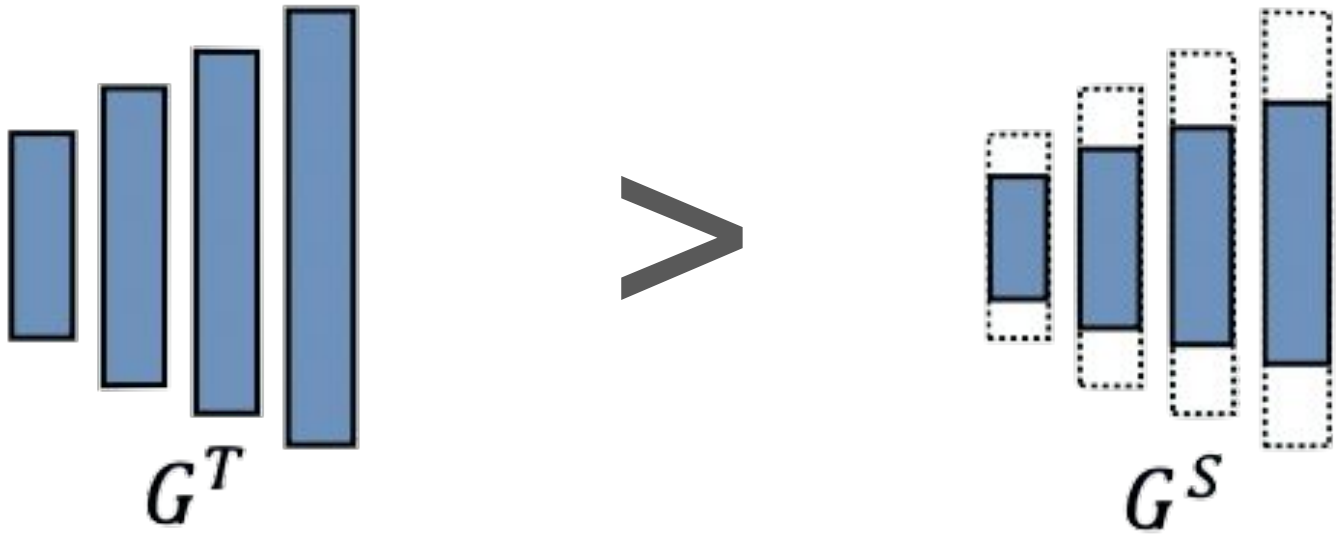
NICKEL





train 초기 단계의 Gs는 랜덤화된 네트워크에 가까움 -> train 초기 단계의 성능이 불안정함!

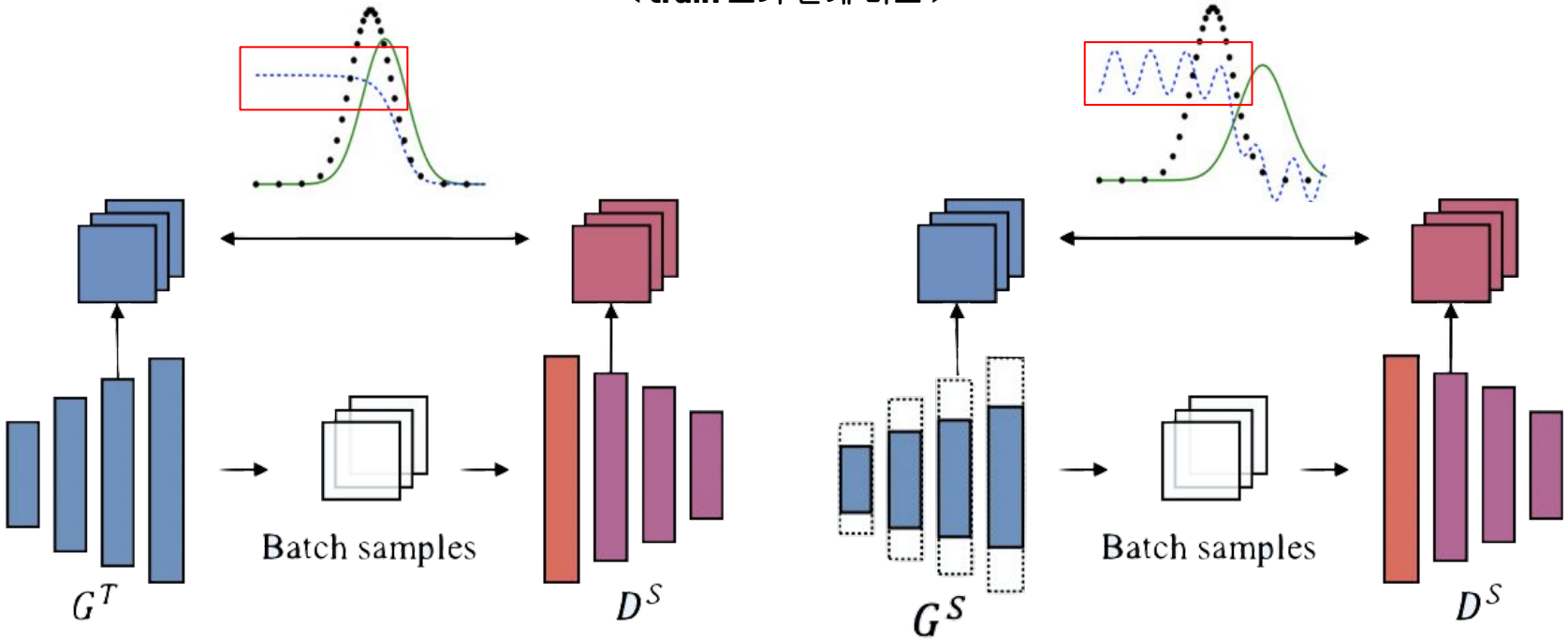
NICKEL



pre-trained 모델을 사용하는 Gt는 train 초기 단계에서 Gs보다 유리함

NICKEL

< train 초기 단계 비교 >



Gt는 train 초기 단계부터 stable하게 Ds에게 풍부한 지식을 제공할 수 있음

NICKEL

$$G^T(z)$$

NICKEL

$$D_i^S(G^T(z))$$

NICKEL

$$G_i^T(z),\ \ D_i^S(G^T(z))$$

Gt와 Ds의 channel shape이
다를 수 있음!

NICKEL

$$G_i^T(z), f_i(D_i^S(G^T(z)))$$

NICKEL

$$\mathcal{L}_{\texttt{NICKEL}} = \sum_{i=1}^{L} d_{\texttt{NICKEL}}(G_i^T(z), f_i(D_i^S(G^T(z))))$$

웨이블릿 loss

NICKEL

$$\mathcal{L}_{\text{NICKEL}} = \sum_{i=1}^{L} d_{\text{NICKEL}}(G_i^T(z), f_i(D_i^S(G^T(z))))$$

웨이블릿 loss

$$LL_i(z_x, z_y) = H_0(z_y^{2^{i-1}})H_0(z_x^{2^{i-1}})LL_{i-1}(z_x, z_y), \quad (1)$$

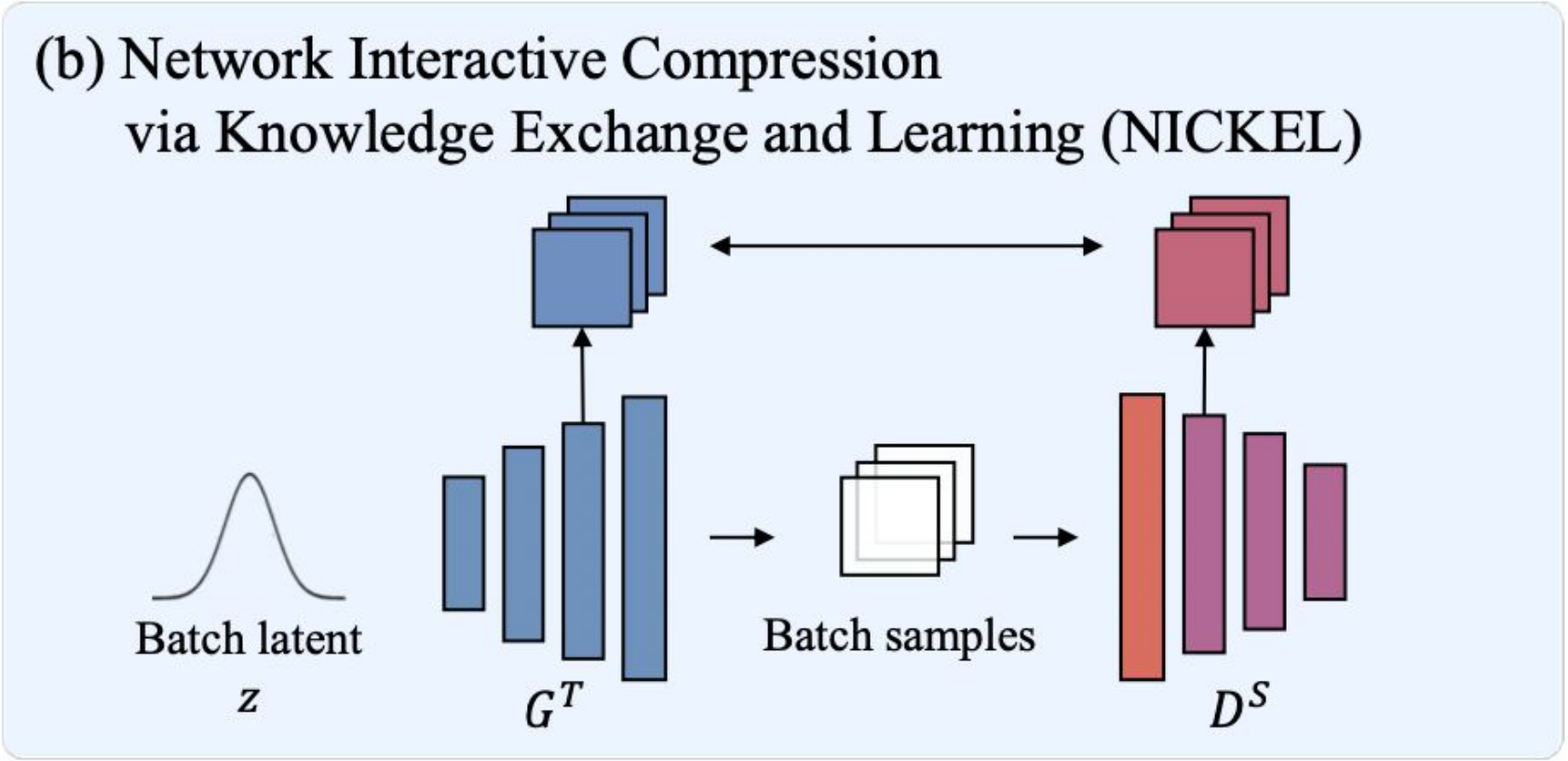
$$LH_i(z_x, z_y) = G_0(z_y^{2^{i-1}})H_0(z_x^{2^{i-1}})LL_{i-1}(z_x, z_y), \quad (2)$$

$$HL_i(z_x, z_y) = H_0(z_y^{2^{i-1}})G_0(z_x^{2^{i-1}})LL_{i-1}(z_x, z_y), \quad (3)$$

$$HH_i(z_x, z_y) = G_0(z_y^{2^{i-1}})G_0(z_x^{2^{i-1}})LL_{i-1}(z_x, z_y), \quad (4)$$

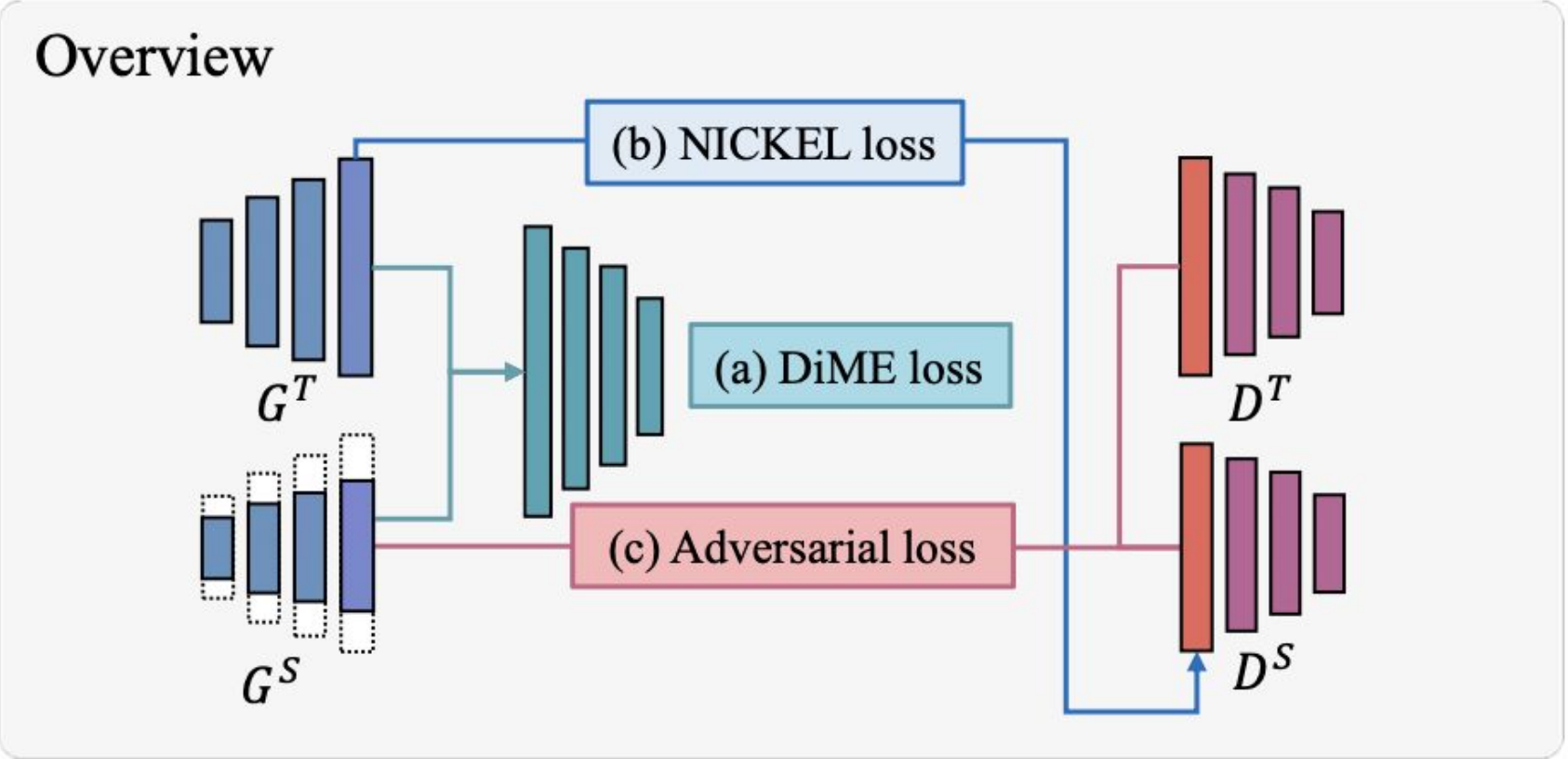
이미지의 미세한 detail을 담고 있는 고주파 영역을 학습하면서 pruning만으로는 떨어지는 detail을 잡아줌

NICKEL


(b) Network Interactive Compression
via Knowledge Exchange and Learning (NICKEL)
Batch latent
z
G^T
Batch samples
D^S

정리

전체 구조


Overview
(b) NICKEL loss
(a) DiME loss
(c) Adversarial loss
G^T
G^S
D^T
D^S

전체 구조

$$\mathcal{L} = \mathcal{L}_{adv} + \underbrace{\lambda_{dino} \cdot \mathcal{L}_{dino} + \lambda_{clip} \cdot \mathcal{L}_{clip}}_{\text{DiME loss}} + \underbrace{\lambda_{\texttt{NICKEL}} \cdot \mathcal{L}_{\texttt{NICKEL}}}_{\text{NICKEL loss}}$$

정리

- DiME

$$\mathcal{L}_{KD} = d_{kd}(G^T(z), G^S(z)) = \mathbb{E}[||\phi(G^T(z)), \phi(G^S(z))||_1]$$

  - Gt -> Gs로 지식 증류!
  - 고차원 이미지 대신 임베딩 공간의 분포 차이를 비교함으로써 계산 비용 절감 & 성능 저하 방지
  - Gt의 global feature를 사용하고 임베딩 공간을 통일시켜 주었기에 샘플링 오류를 줄일 수 있음

- NICKEL

$$\mathcal{L}_{\text{NICKEL}} = \sum_{i=1}^{L} d_{\text{NICKEL}}(G_i^T(z), f_i(D_i^S(G^T(z))))$$

  - Gt -> Ds로 지식 증류!
  - GGDR과 마찬가지로 G에서 D로 지식을 증류하여 효과적으로 성능을 높일 수 있음
  - Gt를 사용함으로써 train 초기 단계에서 훨씬 학습이 안정적임

Experiments

Experiments

Setting - Dataset



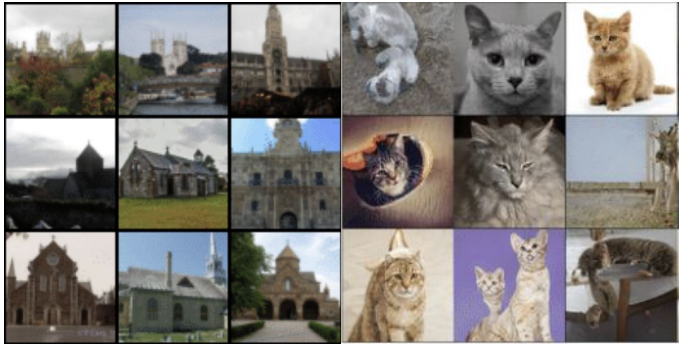
FFHQ



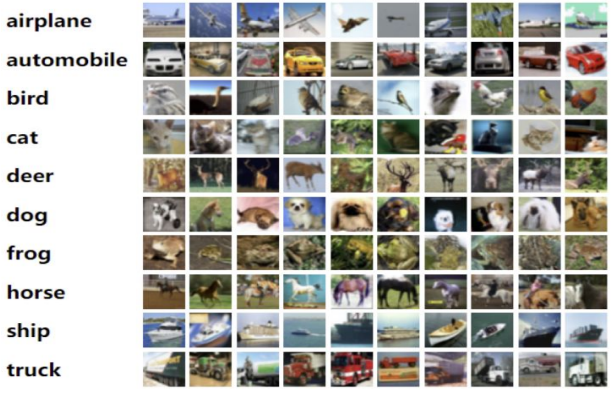
CelebA



AFHQ



LSUN-Church 와 LSUN-CAT



CIFAR-10

Experiments

Setting - 성능 평가 지표

- FID : 실제 이미지와의 품질과 유사성 측정
  (FID 값이 낮을수록 생성된 이미지가 실제 이미지에 가까움)

- Precision : 데이터의 정확도 측정
  (Precision 값이 높을수록 정확도가 높음)

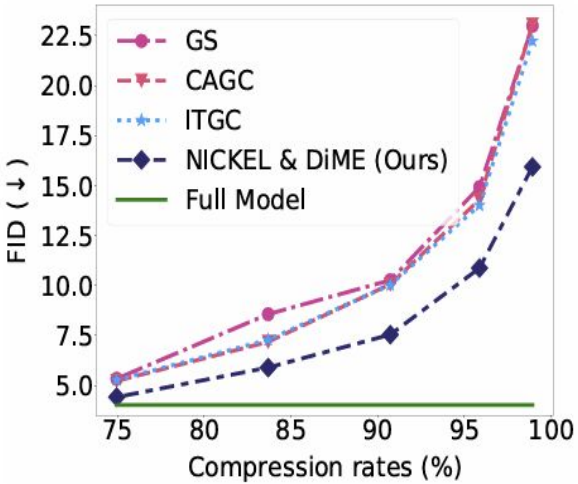
- Recall : 측정
  (Recall 값이 높을수록 다양성이 높음)

$$\text{recall}(\Phi_r, \Phi_g) = \frac{1}{|\Phi_r|} \sum_{\phi_r \in \Phi_r} f(\phi_r, \Phi_g)$$

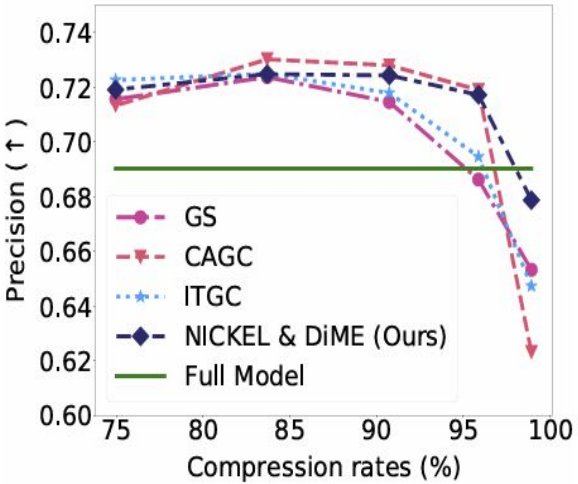
Model	Dataset	Method	#params.	FLOPs	Compression rate	FID↓
StyleGAN2	FFHQ (256×256)	(1) Full model [23]	24.77M	14.90B	-	4.02
		(2) Full model†	30.03M	45.12B	-	4.5
		(3) CAGC† [39]	5.57M	4.12B	90.87%	7.9 → 10.82
		(4) DCP-GAN† [5]				6.35 → 8.93
		CAGC	8.72M	3.73B	74.96%	5.24
		ITGC [21]				5.27
		DiME				5.00
		NICKEL & DiME				**4.42**
		GS [57]	4.96M	1.38B	90.73%	10.26
		CAGC				10.06
		GCC [36]				11.19
		ITGC				10.02
		DiME				8.39
		NICKEL & DiME				**7.43**
		CAGC	2.69M	0.16B	98.92%	23.05
		ITGC				22.23
		DiME				18.32
		NICKEL & DiME				**15.93**
	FFHQ (1024×1024)	Full model	30.37M	74.27B	-	2.74
		Full model†	49.1M	74.3B	-	2.7
		(5) CAGC†	9.2M	7.0B	89.39%	7.6 → 7.53
		(6) DCP-GAN†				5.80 → 5.87
		NICKEL & DiME	5.65M	6.99B	90.59%	**6.41**
	LSUN Church (256×256)	Full model	30.03M	45.12B	-	3.97
		CAGC	5.57M	4.12B	90.87%	4.50
		StyleKD				4.47
		(7) DCP-GAN†				4.87 → 4.82
		NICKEL & DiME				**3.94**
DDPM	LSUN Church (256×256)	Full model†	113.7M	497.4B	-	10.6
		SPDM (t=100)† [7]	63.2M	277.6B	44.19%	13.9

Model	Dataset	Method	#params.	FLOPs	Compression rate	FID↓
SNGAN	CIFAR-10 (32×32)	Full model [41]	4.28M	3.36B	-	17.71
		CAGC [39]	1.20M	0.85B	74.88%	40.45
		ITGC [21]				43.66
		DiME				31.98
		NICKEL & DiME				**23.11**
		CAGC	0.50M	0.31B	90.85%	51.93
		ITGC				59.99
		DiME				36.89
		NICKEL & DiME				**28.27**
BigGAN	CIFAR-10 (32×32)	Full model [1]	8.83M	3.83B	-	10.66
		CAGC	0.89M	0.37B	90.45%	**26.32**
		ITGC				27.78
		NICKEL & DiME				26.66
StyleGAN2	CelebA (128×128)	Full model [23]	24.53M	11.27B	-	2.70
		CAGC	3.08M	0.27B	97.60%	10.05
		ITGC				10.09
		StyleKD				10.41
		NICKEL & DiME				**7.56**
	AFHQ (512×512)	Full model	30.28M	59.67B	-	3.01
		NICKEL & DiME	10.30M	14.94B	74.96%	**3.17**
	LSUN CAT (256×256)	Full model	24.77M	14.90B	-	8.19
		GS	4.96M	1.38B	90.73%	17.11
		CAGC				12.31
		ITGC				12.06
		DiME				11.59
		NICKEL & DiME				**10.80**

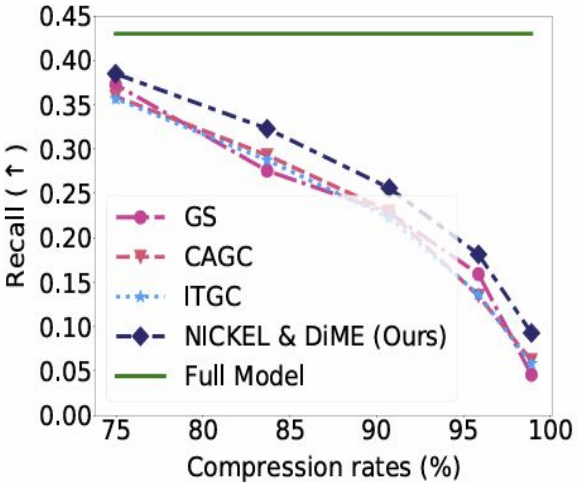
국민대학교 엣지컴퓨팅 B조



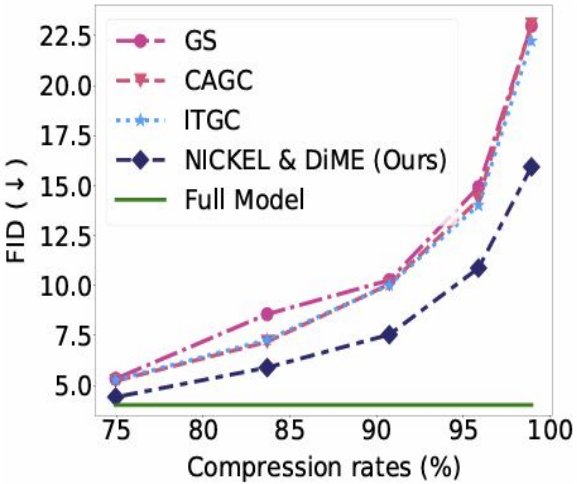
(a) Comparison of FID



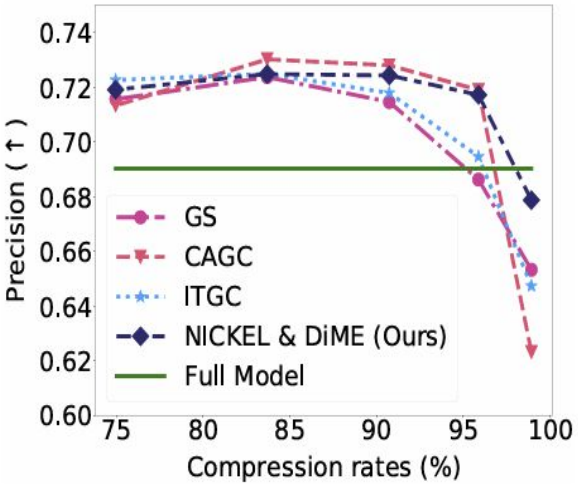
(b) Comparison of Precision



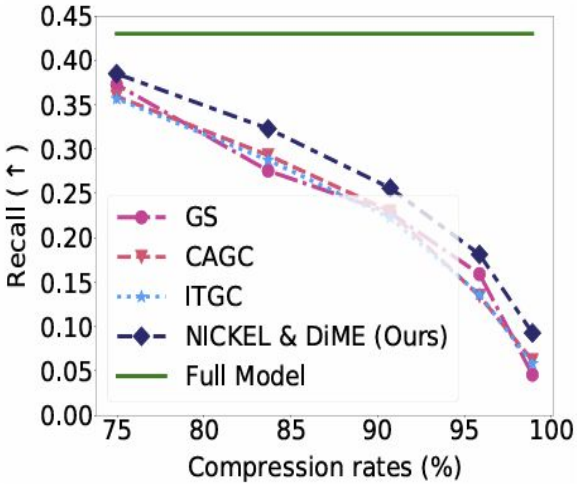
(c) Comparison of Recall



(a) Comparison of FID

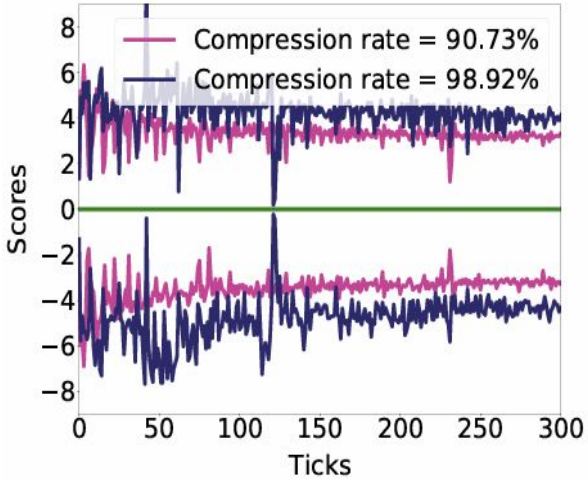


(b) Comparison of Precision

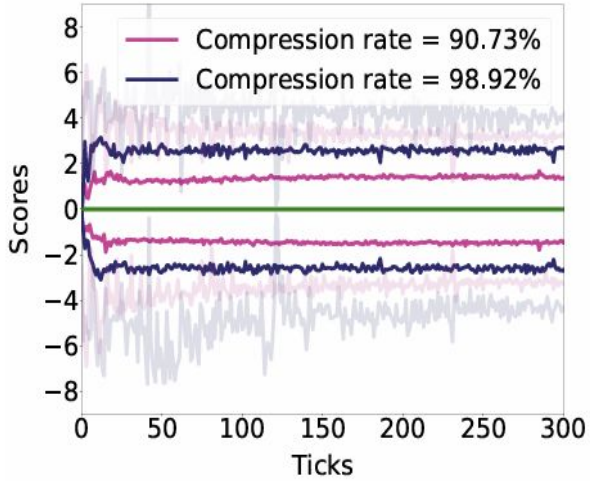


(c) Comparison of Recall

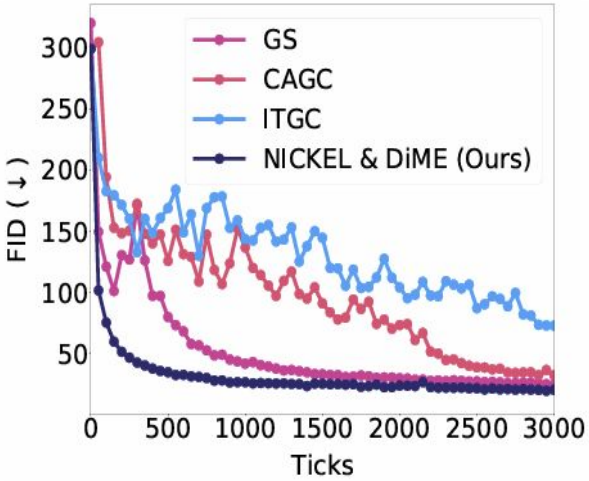
-> 다른 압축 모델보단 **Recall** 성능이 높지만,
전반적으로 모든 모델들이 압축률이 높아질 수록 **Recall** 값이 감소하는 문제가 있음.



(a) Logits of D on ITGC [21]



(b) Logits of D on NICKEL & DiME



(c) Convergence of FID

Model	Dataset	Method	#params.	FLOPs	Compression rate	FID↓
StyleGAN2	FFHQ (256×256)	Full model [23]	24.77M	14.90B	-	4.02
		GS [57]	2.35M	0.05B	99.69%	184.33
		CAGC [39]				186.61
		ITGC [21]				164.92
		NICKEL & DiME				**29.38**

국민대학교 엣지컴퓨팅 B조

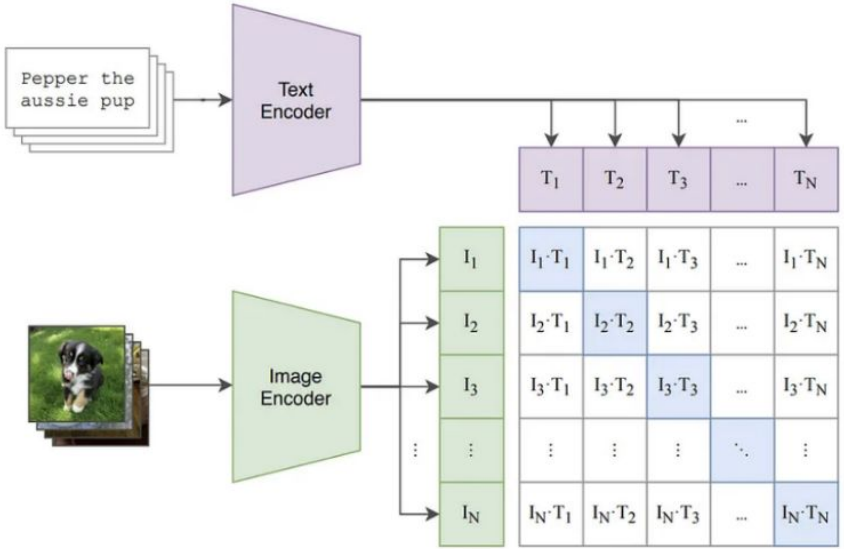


(a) compression rate = 98.92%

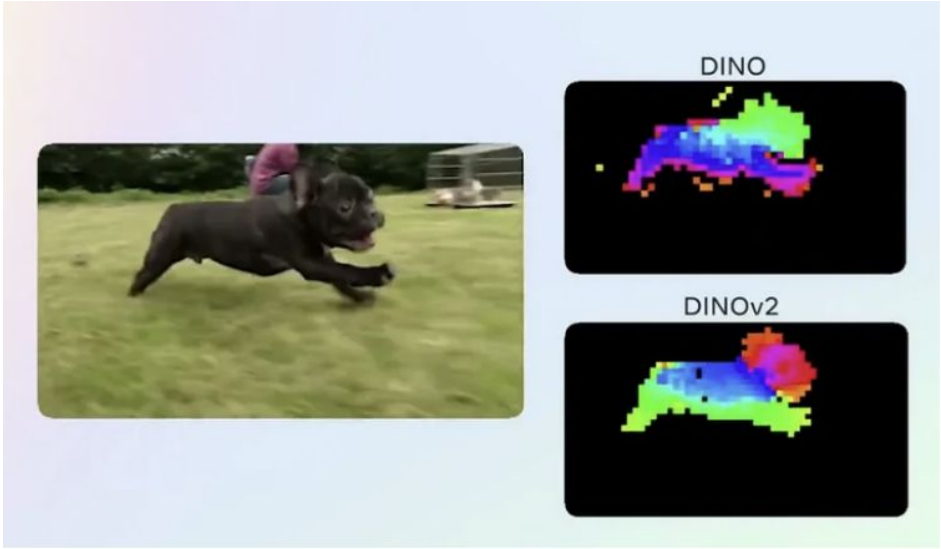


(b) compression rate = 99.69%

Name	Model		Global features	NICKEL	FID↓
	DINO	CLIP			
CLIP embedding w/o global		✓			152.15
DINO embedding w/o global	✓				20.75
DINO embedding	✓		✓		19.60
DiME	✓	✓	✓		18.32
NICKEL & DiME	✓	✓	✓	✓	**15.93**


Pepper the aussie pup
Text Encoder
Image Encoder
T_1 T_2 T_3 ... T_N
I_1 I_2 I_3 ⋮ I_N
$I_1 \cdot T_1$ $I_1 \cdot T_2$ $I_1 \cdot T_3$ ... $I_1 \cdot T_N$
$I_2 \cdot T_1$ $I_2 \cdot T_2$ $I_2 \cdot T_3$ ... $I_2 \cdot T_N$
$I_3 \cdot T_1$ $I_3 \cdot T_2$ $I_3 \cdot T_3$ ... $I_3 \cdot T_N$
$I_N \cdot T_1$ $I_N \cdot T_2$ $I_N \cdot T_3$ ... $I_N \cdot T_N$

CLIP


DINO
DINOv2

DINO

Name	Model		Global features	NICKEL	FID↓
	DINO	CLIP			
CLIP embedding w/o global		✓			152.15
DINO embedding w/o global	✓				20.75
DINO embedding	✓		✓		19.60
DiME	✓	✓	✓		18.32
NICKEL & DiME	✓	✓	✓	✓	**15.93**

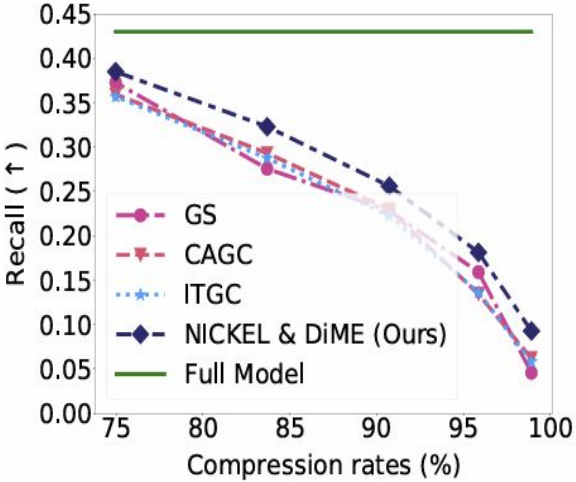
$$\epsilon_{KD} < \epsilon_{teacher} + \epsilon_{student}$$

국민대학교 엣지컴퓨팅 B조

Limitations

- 모든 방법들이 낮은 압축률에서도 **Recall** 성능이 저하되는 것을 통해 **Diversity**가 떨어진다는 한계점을 가진다.

- 향후 극단적인 압축률에서도 다양성과 안정성을 유지할 수 있는 새로운 방법 개발 필요성 제시.



(c) Comparison of Recall

Conclusion

- 이 논문에서는 효율적인 압축을 위한 분포 매칭 방법 (DiME)과

  지식 교환 및 학습을 통한 네트워크 상호작용 압축 (NICKEL)을 제안.

- NICKEL & DiME은 GAN 모델이 극단적인 압축률에서도

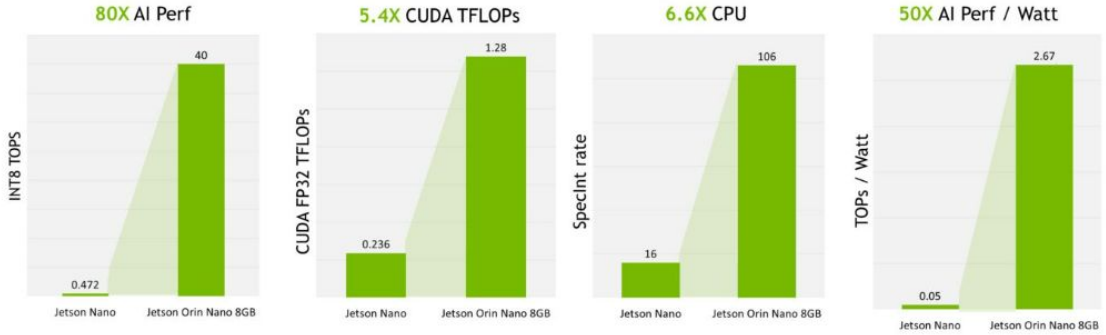
  높은 성능을 낼 수 있는 효과적인 방법.

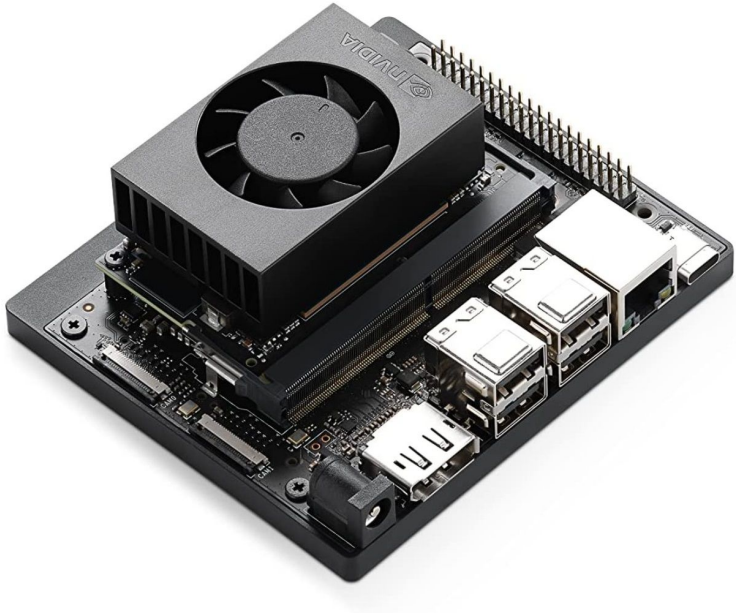
- 자원이 제약된 환경에서도 GAN 모델의 안전성과 우수한 성능을 입증함.

Appendix

Appendix

Jetson Orin Nano로 NICKEL&DiME Inference

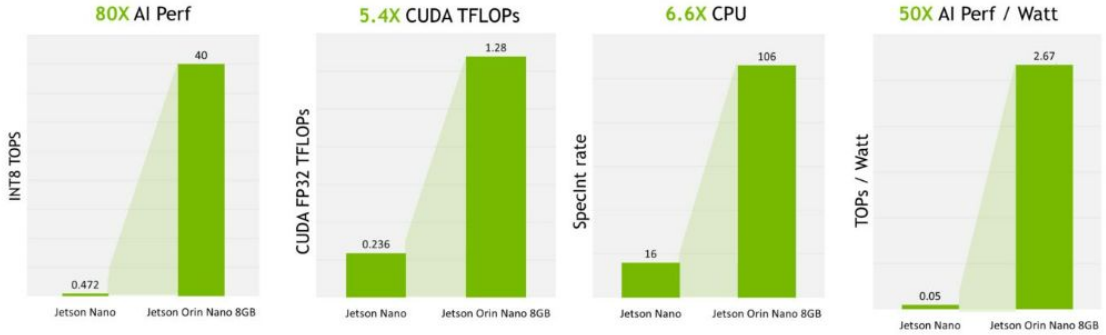


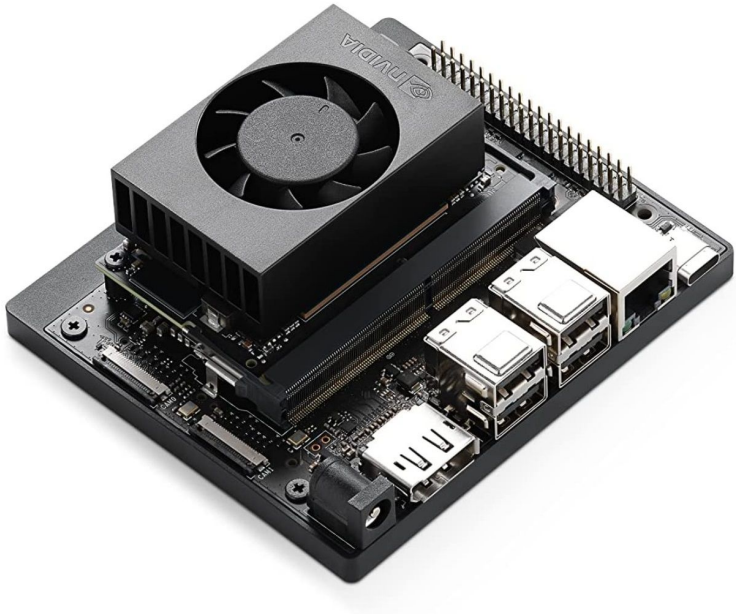


Appendix

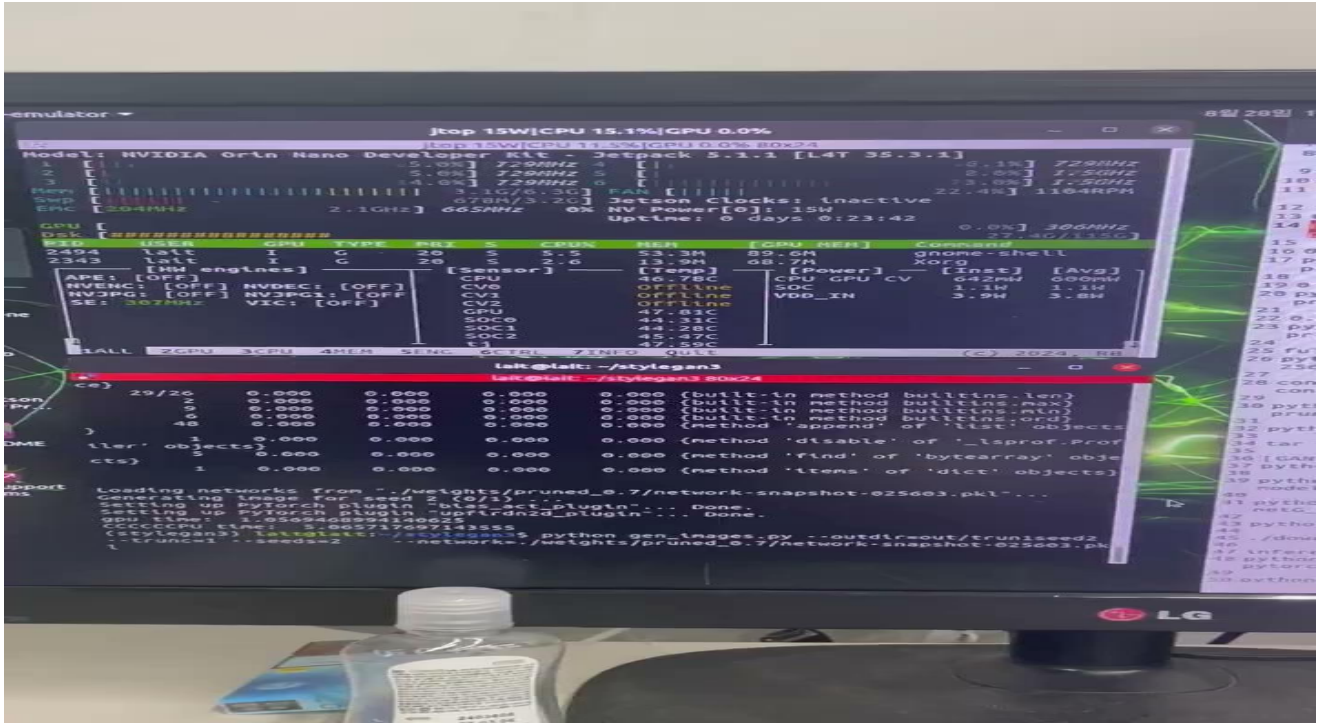
Jetson Orin Nano로 NICKEL&DiME Inference

=> 가격대비 높은 TOPS, GPU&CPU Utility를 가진 임베디드 모델





Appendix


jtop 15W|CPU 15.1%|GPU 0.0%
Model: NVIDIA Orin Nano Developer Kit - Jetpack 5.1.1 [L4T 35.3.1]
Jetson Clocks: inactive
NV Power[0]: 15W
Uptime: 0 days 0:23:42
LG

Appendix

	prunning 0.5	prunning 0.6	prunning 0.7	prunning 0.8	prunning 0.9	prunning 0.95
GPU inference time	0.3278	0.3205	0.3133	0.2974	0.2897	0.292
CPU time	3.3411	3.4389	3.5231	3.4004	3.5187	3.5187
cpu utilization	22.8892	21.6667	23.6667	21.3333	22.3333	19
gpu utilization	100	100	100	100	100	100

Desktop Result in NICKEL&DiME

	pruned 0.5	pruned 0.6	pruned 0.7	pruned 0.8	pruned 0.9	pruned 0.95
GPU inference time	1.3136	1.213	1.1967	1.0909	1.1635	1.1381
CPU time	5.7847	5.268	5.2351	4.966	5.4029	5.3400
cpu utilization	13.4667	10.6	15.3333	12.2333	11.23333	11
gpu utilization	70.9	62.8667	50.9	62.4	58.7	54.8667

Jetson Orin NAno Result in NICKEL&DiME

Appendix

	pruned 0.5	pruned 0.6	pruned 0.8	pruned 0.95	full-model
GPU inference time	3.1346	3.1246	2.9741	2.856	3.5632
CPU time	2.0623	2.3257	1.8022	1.7941	3.1076
cpu utilization	18.3333	16.3333	15.3333	14.7	23.2
gpu utilization	61.7333	62.7	60.0667	56.0333	66.7

Jetson Orin Nano Result in DCP-GAN

	pruned 0.8	pruned 0.95
GPU inference time	51.3649	61.3513
CPU time	8.8267	8.5763
cpu utilization	99.6333	99.5667
gpu utilization	19.2	27.9333

Jetson Orin Nano Result in Content-Aware Compression

Appendix

	pruned 0.5	pruned 0.6	pruned 0.8	pruned 0.95	full-model
GPU inference time	3.1346	3.1246	2.9741	2.856	3.5632
CPU time	2.0623	2.3257	1.8022	1.7941	3.1076
cpu utilization	18.3333	16.3333	15.3333	14.7	23.2
gpu utilization	62.7333	62.7	62.7	26.0333	66.7

Jetson Orin Nano Result in DCP-GAN

모든 모델은 StyleGAN2 기반으로 모델 Architecture가 같음

	pruned 0.8	pruned 0.95
GPU inference time	51.3649	61.3513
CPU time	8.8267	8.5763
cpu utilization	99.6333	99.5667
gpu utilization	19.2	27.9333

Jetson Orin Nano Result in Content-Aware Compression

Appendix

(1) 본 모델은 하드웨어 리소스가 제한된 임베디드 플랫폼에서도 높은 성능과 안정적으로 추론이 가능함

(2) 또한 styleGAN2를 기반으로 하는 다른 모델에 비해 inference 시간이 빠르고 안정적으로 리소스를 사용하는 것을 확인 함

Q & A